Database System Performance on Recursive Queries

John Idogun

Research Proficiency Exam

Computer Science Department
Stony Brook University, State University of New York

January 23, 2026

Committee

Prof. Scott D. Stoller
Prof. David S. Warren
Computer Science Department

Advisor

Prof. Yanhong A. Liu
Computer Science Department

Abstract

Recursive queries are fundamental for analyzing complex graphs and relationships in applications such as determining ancestors in genealogy, resolving access control in hierarchical user roles, mapping prerequisites for graduation, and tracing bank transaction activities. Despite the presence of support for recursion in many database systems, their underlying implementations, performance characteristics, and correctness guarantees vary significantly. This study presents a systematic analysis and performance evaluation of recursive queries across a curated selection of dominant database architectures: traditional SQL (PostgreSQL, MariaDB, DuckDB), NoSQL (MongoDB, Neo4j), and NewSQL (SingleStore, CockroachDB). These systems were chosen for their industry adoption, performance claims, and availability for empirical testing. For a logic programming baseline, the highly optimized XSB Prolog system was also included.

We conduct a comprehensive evaluation of left, right, and non-linear (double) recursion across twelve distinct graph topologies. Our findings reveal a fragmented landscape of support and correctness. PostgreSQL and CockroachDB are limited to linear recursion. MariaDB supports non-linear recursion, but only when a standard-compliance flag is disabled. SingleStore's support is severely restricted, precluding most practical recursive use cases. Notably, while DuckDB supports non-linear recursion, our investigation found it returned incorrect results for 7 of the 12 tested graph structures, a critical finding for practitioners relying on it for complex graph analytics [Ido24]. In performance benchmarks for linear recursion, DuckDB consistently demonstrated the lowest elapsed times, outperforming its SQL counterparts.

Contents

1 Introduction **4**

2 A Comparative Analysis of Database Architectures **5**
2.1 Relational Database Management Systems (RDBMS) 5
2.2 NoSQL Databases 6
2.3 NewSQL Databases 7
2.4 Systems Under Evaluation 8

3 Transitive Closure in Database Systems **9**
3.1 The Relational Database Approach: Transitive Closure via Iterative Joins . . 9
3.2 The Document Store Approach: Fixed-Algorithm Traversal 11
3.3 The Graph-Native Approach: Built-In Path Traversal 12

4 Experimental Analysis and Discussion **13**
4.1 Performance Measurement Methodology 13
4.2 Justification for System Exclusions from Performance Benchmarking 14
4.3 Performance Analysis Across Graph Topologies 15
4.4 Architectural Implications and Overall Findings 29

5 Related work **31**
5.1 Conclusion, limitations, and future works 32

References **33**

1 Introduction

Recursive queries are fundamental for analyzing complex graphs and relationships. Their applications are critical and widespread, ranging from resolving access control in hierarchical user roles and mapping academic prerequisites, to tracing financial transactions and identifying optimal paths in network routing algorithms. Furthermore, recursive queries are crucial for analyzing social networks, managing complex inventory systems in manufacturing, and navigating organizational structures to compute indirect reporting lines.

Database benchmarking has a long and storied history, with extensive literature dedicated to performance measurement [BDT83], particularly through the influential Transaction Processing Performance Council (TPC) benchmarks [Gra92, AFB15, ABD+16, FH17, OB17]. However, these standard benchmarks have historically given little attention to recursive queries [BW19, KJN22], a feature that was formally added to the SQL standard only in 1999 [EM99, MS01, Ord05, Ord10, BWS14]. Even today, robust support for recursion is inconsistent across systems. Many databases primarily support the simpler linear form of recursion, and the optimization of recursive queries remains a significant challenge for modern query planners [Ord05, Ord10, BEA+20]. Ensuring the correctness of these complex operations is an equally formidable challenge, as our findings regarding the analytical database DuckDB demonstrate [RM19a, Ido24].

While logic programming systems, particularly by subsuming Datalog, are inherently well-suited for recursion [LFWK09, BW19], they are generally not designed for the concurrent, persistent transactional workloads common in enterprise systems and often lack the sophisticated parallelism and caching found in mainstream database systems. This leaves a critical gap in understanding how different modern data query engines —each with distinct design philosophies—handle the performance and correctness challenges of this powerful query paradigm.

This paper presents a systematic, empirical comparison of database systems spanning the dominant modern database categories: traditional SQL (PostgreSQL, MariaDB, DuckDB), NoSQL (MongoDB, Neo4j), and NewSQL (SingleStore, CockroachDB). For a specialized baseline, the highly-optimized logic programming system XSB Prolog is also included due to its reputed efficiency in handling recursive queries [SW12]. We evaluate all three fundamental types of recursion (left, right, and non-linear/double) across 12 distinct graph topologies, conducting both performance analysis and evaluation.

The main contributions of this paper include:

- Detailed and repeatable running time measurements encompassing various aspects such as table creation, data loading, query optimization, query execution, and writing results to file.
- A comprehensive comparison of the three main categories of database systems —SQL, NoSQL, and NewSQL —on recursive workloads.
- Precise performance predictions for each system for each recursion variant across all graph types, regardless of the input and output sizes.

Our findings reveal that PostgreSQL and CockroachDB support only linear recursion; thus, double recursion was not feasible. DuckDB supports double recursion but returned incorrect results in 7 out of 12 graph types with varying node numbers. MariaDB also supports non-linear recursion, but only when its standard CTE compliance is turned off. SingleStore does not support recursion in its recursive common table expression. For both left and right recursion, DuckDB demonstrated superior performance in elapsed times compared to PostgreSQL, CockroachDB, and MariaDB across the considered node sets.

2 A Comparative Analysis of Database Architectures

We analyze three dominant architectural paradigms in database management systems (DBMSs), each representing a different philosophy for data management, consistency, and performance. The systems selected for this study are not merely a sample of popular products; they are exemplars of these distinct database categories, chosen to illuminate the fundamental trade-offs that impact the performance and correctness of recursive queries.

2.1 Relational Database Management Systems (RDBMS)

Traditional RDBMSs are founded based on the principles of the relational model and are characterized by their strict schema enforcement and robust transactional guarantees, commonly known as ACID properties (Atomicity, Consistency, Isolation, Durability) [HR83]. The introduction of recursive Common Table Expressions (CTEs) into the SQL standard was a significant extension to this model, allowing these systems to handle hierarchical and graph-like data natively [WZ02, Boe20].

PostgreSQL As one of the most advanced open-source RDBMSs, PostgreSQL serves as a baseline for standards-compliant SQL recursion [SWKH76, DCH05]. It is renowned for its extensibility, robust support for complex queries, and a rich variety of index types (e.g., B-tree, GIN, GiST) that can be leveraged by the query optimizer [GWD21]. PostgreSQL implements a process-per-connection model for strong isolation and fault tolerance and uses Multi-Version Concurrency Control (MVCC), which is particularly advantageous for the mixed workloads that often include long-running analytical queries [RQ08]. By providing each transaction with a consistent snapshot of the database, MVCC avoids the read/write locks that could otherwise stall recursive queries in a highly concurrent environment [LJA+23]. Its implementation of recursive CTEs adheres strictly to the SQL standard's definition of linear recursion, where the recursive term may only reference the CTE once, and follows an iterative evaluation model that mirrors semi-naive evaluation [FGS+21].

MariaDB A fork of MySQL, MariaDB is engineered for high performance and compatibility, but it distinguishes itself with a more flexible approach to recursion [TPLH16]. While its default configuration is standard-compliant, MariaDB provides the option to support non-linear recursion—where a CTE can be joined with itself multiple times in the recursive

step—by setting the `standard_compliant_cte` variable to 0 [SN17]. This non-standard extension enables more direct and sometimes more efficient expressions for certain algorithms, such as finding same-generation cousins. This feature, however, alters the evaluation semantics: in each iteration, the recursive term operates on all rows produced so far, not just the new rows from the previous iteration, a critical distinction for query authors [WZ02]. This feature is further enhanced by its pluggable storage engine architecture and built-in support for Galera Cluster for synchronous multi-master replication [GV20].

DuckDB DuckDB represents a modern class of in-process analytical RDBMSs (an analytical RDBMS that runs in the same Operating System process as the application that uses it) designed for high-performance OLAP workloads [RM19a]. Its architecture is fundamentally different from traditional row-oriented OLTP systems, employing columnar storage and a vectorized query execution engine [SA16, MDP23]. This model processes data in batches (vectors) rather than row-by-row, which can dramatically accelerate the scan-and-join loops inherent in recursive CTEs [BMPB21], and recursive CTEs are executed via semi-naive evaluation [Ido24]. However, this architecture also has distinct trade-offs. As observed in performance analyses, workloads that produce only a single row per iteration can suffer from the overhead of the vectorized engine, leading to slower performance than row-oriented systems like PostgreSQL [RM19a]. DuckDB has also introduced novel, non-standard extensions such as `USING KEY` to optimize stateful graph traversals by allowing updates to intermediate results, which can significantly reduce memory consumption and improve performance for algorithms like shortest paths [Duc24b].

2.2 NoSQL Databases

The NoSQL paradigm emerged to address the scalability and flexibility limitations of traditional RDBMSs, particularly for large-scale, unstructured, or semi-structured data. These systems typically prioritize availability and horizontal scalability (the scalability where a system's capacity is increased by adding more machines or nodes), often relaxing strict ACID guarantees in favor of the BASE (Basically Available, Soft state, Eventually consistent) model [BB16, ZMB17].

MongoDB As a leading document-oriented database, MongoDB stores data in flexible, JSON-like documents, which allows for a schema-less design well-suited for rapid, agile development [Dah17]. Its approach to recursion is not through declarative SQL but via an imperative operator within its aggregation framework: `$graphLookup` [Mon25b]. This operator performs a recursive search on a collection, traversing connections between documents. However, because it is an aggregation pipeline stage rather than a globally optimizable declarative construct, its performance is highly dependent on the structure of the pipeline and the effective use of filters to constrain the search space [TPLH16]. Furthermore, `$graphLookup` is subject to a 100-megabyte memory limit and does not guarantee the order of results, requiring subsequent sorting operations if order is needed [Mon25b].

Neo4j In contrast, Neo4j is a native graph database architected from the ground up to manage and query highly connected data [AAB+23]. Its core architectural advantage is index-free adjacency, where each node stores direct pointers to its adjacent nodes and relationships [Web12]. This design allows for extremely fast traversals, as navigating from one node to the next is a constant-time operation, akin to dereferencing a pointer, rather than an expensive index lookup and join operation [RWE15a]. Consequently, the performance of deep, multi-hop queries in Neo4j is proportional to the size of the subgraph being explored, not the size of the overall dataset, giving it a significant asymptotic advantage over RDBMSs for traversal-heavy workloads [AAB+23]. However, this specialization comes with trade-offs, including slower performance for global bulk-scan queries where relational indexes are more efficient [VMZ+10a].

2.3 NewSQL Databases

NewSQL databases aim to provide a synthesis of the preceding paradigms, combining the horizontal scalability and high performance of NoSQL systems with the ACID guarantees and SQL interface of traditional RDBMSs [PA16].

CockroachDB CockroachDB is a distributed SQL database designed for global consistency, horizontal scalability, and resilience to failures [TSM+20]. It achieves this by partitioning data into ranges and distributing them across a cluster, using the Raft consensus protocol to ensure consistency [OO14]. While it offers PostgreSQL wire compatibility (referring to the database's ability to communicate using the same network protocol as PostgreSQL), executing iterative queries in this distributed environment presents unique challenges. Each step of a recursive CTE may require data to be shuffled between distributed nodes via RPCs, incurring network latency that can become a significant bottleneck [TSM+20]. This overhead is a primary consideration for recursive query performance in any distributed SQL system. Furthermore, CockroachDB imposes certain limitations that affect complex applications; for instance, user-defined functions (UDFs) cannot be recursive, which prevents developers from encapsulating reusable recursive logic within a function [Coc25b].

SingleStore Formerly MemSQL, SingleStore is a distributed, in-memory-optimized database designed for hybrid transactional/analytical processing (HTAP) workloads [CTW+21]. While it supports the `WITH RECURSIVE` syntax, its practical utility for complex recursion is severely curtailed by a number of significant limitations in the recursive term of a CTE. Operations such as `GROUP BY`, aggregates (e.g., `COUNT`, `MAX`), `SELECT DISTINCT`, `UNION` and `OUTER JOINS (LEFT and RIGHT)`, among others are explicitly disallowed as of version 8.7 [Sin24]. These restrictions make it impossible to perform many common recursive tasks directly within the query, such as finding the shortest path using aggregation or counting descendants in a hierarchy. This suggests an implementation that prioritizes basic reachability but avoids the complexities of optimizing and executing more advanced recursive logic in a distributed environment, effectively rendering it unsuitable for many of the analytical use cases this paper investigates.

2.4 Systems Under Evaluation

The database systems for this study were selected to represent the architectural diversity of the current state of the art while meeting a set of practical criteria. Each chosen system is either a leader or a significant innovator in its category, has a strong professional outlook [Inc24], is open-source or provides a free license for testing, and offers some form of support for recursive queries. This selection, tabulated in Table 1, allows for a comprehensive and comparative analysis of how different architectural philosophies address the challenges of recursive query processing.

System	**Architectural Paradigm**	**Core Execution Model**	**Recursion Support Model**	**Key Limitations / Trade-offs**
PostgreSQL	Single-Node RDBMS (Object-Relational)	Process-per-connection, Row-oriented, MVCC	SQL-standard recursive CTEs (linear anchor/step)	Lacks support for non-linear recursion; performance can degrade with very deep traversals and no delta optimization.
MariaDB	Single-Node RDBMS	Pluggable Storage Engines (Row or Column-oriented)	Linear recursive CTEs; non-linear via session flag	Non-linear recursion has different evaluation semantics (uses all prior rows, not just delta).
DuckDB	In-process Analytical RDBMS	Vectorized, Columnar (Column-oriented), In-process	Linear recursive CTEs with extensions (USING KEY)	High overhead on single-row iterations; documented correctness bugs with non-linear recursion.
MongoDB	NoSQL (Document Store)	Aggregation Pipeline (Imperative, Stage-Oriented)	Recursive aggregation stage (`$graphLookup`)	Depth and memory constraints; lacks recursion strategy control; performance scales poorly.
Neo4j	NoSQL (Native Graph Database)	Index-Free Adjacency, Traversal-oriented	Cypher supports declarative variable-length path patterns for traversal recursion	Asymptotic performance advantage on deep traversals; slower on bulk scans; lacks recursion strategy control
CockroachDB	NewSQL (Distributed SQL)	Distributed KV-store with SQL compatibility layer	SQL-standard recursive CTEs (linear recursion, PostgreSQL-compatible)	Each iteration may incur network latency in distributed execution; recursive procedural UDFs are unsupported.
SingleStore	NewSQL (Distributed HTAP)	In-memory optimized, Rowstore and Columnstore	Support for recursive CTEs with notable limitations	Only SELECT statements allowed in recursion; restrictions on aggregates, `DISTINCT`, `GROUP BY`, `UNION`, outer joins; engine variable configuration required.

Table 1: Architectural and Feature Comparison of Evaluated Database Systems.

3 Transitive Closure in Database Systems

The computation of transitive closure is a canonical benchmark for recursive query processing, as it represents a class of graph traversal problems that cannot be expressed in standard relational algebra [AU79]. Different database architectures provide fundamentally different methods for computing transitive closure, each with significant implications for query expression, performance, and correctness. We analyze the implementation of transitive closure across the selected systems, focusing on the three primary variants of linear and non-linear recursion.

Given a table `edge` of two columns `x` and `y` representing directed edges from vertex `x` to vertex `y`, the transitive closure of the edges is a table `tc` of two columns `x` and `y` representing that there is a path from vertex `x` to vertex `y` following one or more edges.

3.1 The Relational Database Approach: Transitive Closure via Iterative Joins

In SQL-based systems, transitive closure is implemented using recursive Common Table Expressions (CTEs), which provide a syntax for realizing a least fixed-point computation [Mel03]. The evaluation model is inherently iterative: a non-recursive (base) case seeds the computation, and a recursive step repeatedly joins the results of the previous iteration with base relations until a fixed point is reached and no new tuples are generated [SQL99, Gra93].

Linear Recursion The most common forms of transitive closure are linear, meaning the recursive term contains only a single self-reference to the CTE. The structure of this join determines the evaluation strategy.

Right Recursion:

```
WITH RECURSIVE tc AS (
    SELECT x, y FROM edge
    UNION
    SELECT edge.x, tc.y FROM edge JOIN tc ON edge.y = tc.x
)
SELECT * FROM tc;
```

In this formulation, the recursive relation `tc` appears on the right side of the join. This structure corresponds to a forward-chaining evaluation. In each iteration, the algorithm takes newly discovered paths (in `tc`) and extends them by one hop by joining with the edge table on the left. This is conceptually equivalent to finding paths of length $k+1$ by appending an edge to the beginning of known paths of length k.

Left Recursion:

```
WITH RECURSIVE tc AS (
    SELECT x, y FROM edge
```

```
    UNION
    SELECT tc.x, edge.y FROM tc JOIN edge ON tc.y = edge.x
)
SELECT * FROM tc;
```

Conversely, with the recursive relation on the left side of the join, the query performs a backward-chaining evaluation. It extends existing paths by one hop on the right. While logically equivalent for computing the full transitive closure, the performance of left versus right recursion can differ based on the query planner's choice of join algorithms (e.g., hash join vs. nested loop) and which relation is used as the build-side versus the probe-side in each iteration [Gra93].

Non-Linear Recursion and Its Challenges A more expressive, but computationally intensive, approach is non-linear (or double) recursion, where the recursive relation is joined with itself.

```
WITH RECURSIVE tc AS (
    SELECT x, y FROM edge
    UNION
    SELECT tc1.x, tc2.y FROM tc AS tc1 JOIN tc AS tc2 ON tc1.y = tc2.x
)
SELECT * FROM tc;
```

This query computes new paths by composing any two existing paths where the end of the first path connects to the start of the second. This approach can converge in fewer iterations (logarithmically in the length of the longest path), but each iteration is significantly more expensive due to the quadratic complexity of the self-join. This computational cost and the difficulty of creating optimal query plans are primary reasons why most standard-compliant RDBMSs, including PostgreSQL and CockroachDB, explicitly forbid non-linear recursion [Pos25, Coc25a].

MariaDB is a notable exception, permitting non-linear recursion when the session variable `@@standard_compliant_cte` is set to 0 [SN17]. However, this changes the evaluation semantics; in each iteration, the recursive join is performed over all tuples generated so far, not just those from the previous iteration [SN17].

DuckDB also supports the non-linear form, but our experiments revealed significant correctness issues. For 7 of the 12 graph types tested, the query returned incomplete result sets. For example, on a 20-node binary tree, the query failed to discover paths that required composing two derived paths (e.g., a path of length 2 with another path of length 2), instead only composing derived paths with base edges. This bug, which has been reported [Ido24], underscores the engineering complexity of correctly implementing non-linear recursion, particularly in novel vectorized execution engines.

3.2 The Document Store Approach: Fixed-Algorithm Traversal

Document databases like MongoDB lack a native declarative syntax for recursion but can emulate it using imperative aggregation pipelines.

MongoDB and $graphLookup MongoDB computes transitive closure using the `$graphLookup` operator, which employs a breadth-first search (BFS) algorithm for graph traversal. Given a binary relation $R(x, y)$ represented as an edge collection, the operator computes the transitive closure R^+ by discovering all reachable pairs through iterative path expansion.

Unlike SQL systems where recursion strategy can be explicitly controlled through query structure, MongoDB's `$graphLookup` uses a predetermined BFS algorithm. The operator does not distinguish between left, right, or double recursion—all variations produce identical execution plans and results. A typical implementation is:

```
db.edge.aggregate([
  {
    $graphLookup: {
      from: "edge",
      startWith: "$x",
      connectFromField: "y",
      connectToField: "x",
      as: "paths",
      restrictSearchWithMatch: {}
    }
  },
  { $unwind: "$paths" },
  { $project: {
      _id: 0,
      x: "$x",
      y: "$paths.y"
    }
  },
  { $group: {
      _id: { x: "$x", y: "$y" },
      x: { $first: "$x" },
      y: { $first: "$y" }
    }
  },
  { $project: { _id: 0, x: 1, y: 1 } },
  { $out: "transitive_closure" }
], { allowDiskUse: true })
```

The pipeline proceeds in five stages:

- *Graph Traversal*—for each document with field x, the operator performs BFS by matching `connectFromField` (y) to `connectToField` (x) in subsequent iterations, storing discovered paths in an array;
- *Path Unwinding*—the `$unwind` operator flattens the paths array to create individual tuples for each reachable pair;
- *Projection*—relevant fields (x, y) representing transitive closure pairs are extracted;
- *Deduplication*—the `$group` operator with a composite key eliminates duplicate tuples, ensuring each pair appears only once;
- *Materialization*—results are written to an output collection via `$out`.

This approach is fundamentally procedural: it performs the traversal for each starting document using a fixed BFS strategy, unnests the resulting paths, and then deduplicates. The lack of recursion strategy control makes MongoDB incomparable to systems where different recursion formulations yield different performance characteristics. Additionally, this approach is subject to significant operational limitations, including a 100 MB memory limit per aggregation stage [Mon25a] and per-stage rather than global optimization [Mon25b]. The `allowDiskUse` option enables disk spillage for intermediate results exceeding available memory, though this can lead to suboptimal performance compared to declarative query optimizers that can reorder and optimize operations holistically.

3.3 The Graph-Native Approach: Built-In Path Traversal

Graph databases are architecturally optimized for traversal, providing built-in operations for path enumeration.

Neo4j and Variable-Length Paths Neo4j computes transitive closure using variable-length path patterns in its Cypher query language. Given a graph with nodes and directed edges, the transitive closure R^+ is expressed as:

```
MATCH (start:Node)-[:EDGE*1..]->(end:Node)
RETURN DISTINCT start.id AS x, end.id AS y;
```

The `[:EDGE*1..]` pattern syntax instructs the engine to find paths of arbitrary length between nodes. The query proceeds as follows:

- *Path Enumeration*—the engine traverses the graph to discover all paths of length ≥ 1 from each starting node to reachable nodes;
- *Path Extraction*—for each discovered path, the start and end node identifiers are extracted;

- *Deduplication*—the `DISTINCT` operator eliminates duplicate (x, y) pairs, producing the final transitive closure result.

Like MongoDB's `$graphLookup`, this operation does not support different recursion strategies; the same pattern is used regardless of whether one conceptually desires left, right, or double recursion. The evaluation is performed not through iterative joins but by direct pointer chasing in the graph structure, leveraging Neo4j's index-free adjacency architecture where each node stores direct references to its adjacent edges and nodes [RWE15b, Neo25a]. This native graph traversal approach eliminates the need for index lookups during edge traversal.

However, this approach suffers from a fundamental performance limitation. As noted by Brass and Stephan [BW19], Neo4j's variable-length path matching enumerates *all* paths between nodes before deduplicating with `DISTINCT`, rather than computing reachability directly. On dense or cyclic graphs, the number of paths grows exponentially with path length, making this approach prohibitively expensive. For example, in a complete graph with n nodes, there are $O(n!)$ paths of varying lengths between nodes, yet the transitive closure contains only n^2 unique pairs. Neo4j must enumerate the exponential number of paths before reducing to the polynomial result. This architectural choice—optimizing for path enumeration rather than reachability queries—fundamentally limits Neo4j's suitability for computing transitive closure on arbitrary graphs [BW19]. While the index-free adjacency model provides efficient local traversals, the path-enumeration-then-deduplication strategy dominates performance on graphs where the path count significantly exceeds the reachability count.

4 Experimental Analysis and Discussion

4.1 Performance Measurement Methodology

We conducted a comprehensive performance analysis of recursive query execution across a curated set of database systems and logic programming engines over a diverse of graph topologies (graph kinds as seen in [LSIT25, BW19]). The systems evaluated include PostgreSQL, MariaDB, DuckDB, CockroachDB, SingleStore, MongoDB, Neo4j, and the logic programming system XSB.

Each database system utilized its optimal Python driver to ensure efficient query execution with minimal connection overhead: psycopg2 [DG24] for PostgreSQL and CockroachDB, mysqlclient [tea25] for MariaDB, the native duckdb driver [Duc24a] for DuckDB, pymongo [Mon24] for MongoDB, and neo4j [Neo24] for Neo4j. For XSB, we used the approach discussed in [LSIT25].

Performance was primarily measured in terms of elapsed (real) time across all systems. This approach was necessary because most RDBMSs, including PostgreSQL and MariaDB, do not expose per-query CPU time through standard client drivers, making elapsed time the only consistent metric for cross-system comparison. Conversely, CPU time was reported for XSB and DuckDB, since their expressiveness allowed such. For these single-node, compute-bound workloads, the close correlation observed between CPU and elapsed times for XSB

and DuckDB suggests that elapsed time serves as a reasonable proxy for overall performance. For each experiment, we report the average time over 5 runs.

All measurements were taken on an Apple M3 Pro chip with an 11-Core CPU, 18 GB of unified memory, and a 512GB SSD running macOS Sonoma 14.5. The software versions used were PostgreSQL 16.3, MariaDB 15.2, DuckDB 1.0.0, Neo4j 5.20.0, MongoDB 7.0.11, CockroachDB 24.1.1, and XSB 5.0.0.

4.2 Justification for System Exclusions from Performance Benchmarking

A crucial part of our methodology was an initial evaluation of all selected systems for correctness and baseline performance. This preliminary phase revealed that several systems were not suitable for a direct, scaled performance comparison of transitive closure. These exclusions are themselves significant findings of this study.

- **SingleStore:** This system was excluded due to severe functional limitations in its recursive CTE implementation. The official documentation confirms that the recursive term of a CTE explicitly disallows essential operations such as `GROUP BY`, aggregate functions, `SELECT DISTINCT`, `UNION`, and outer joins [Sin25]. These restrictions make it impossible to perform many common recursive tasks, including several transitive closure variants, rendering it unsuitable for this benchmark.

- **MongoDB:** This system was excluded due to both architectural limitations and prohibitive performance scaling. The `$graphLookup` operator, MongoDB's primary mechanism for recursion, supports neither alternative recursion strategies (left, right, or double recursion) nor user-defined evaluation orders. Unlike Datalog systems and SQL engines with recursive CTEs that permit control over rule evaluation, `$graphLookup` uses a fixed breadth-first search algorithm that computes the full transitive closure without regard to recursion strategy [Mon25b]. This design prevents direct comparability with systems where recursion order is a performance variable. Furthermore, `$graphLookup` is implemented as an imperative aggregation stage, not a declarative set-based operation. Because each stage of the pipeline is subject to a 100-megabyte memory limit [Mon25a, Mon25b], the operator exhibits steep performance degradation under recursive load. For example, in our tests, processing a 100-node complete graph required over 1,500 seconds, underscoring its unsuitability for bulk-recursive workloads.

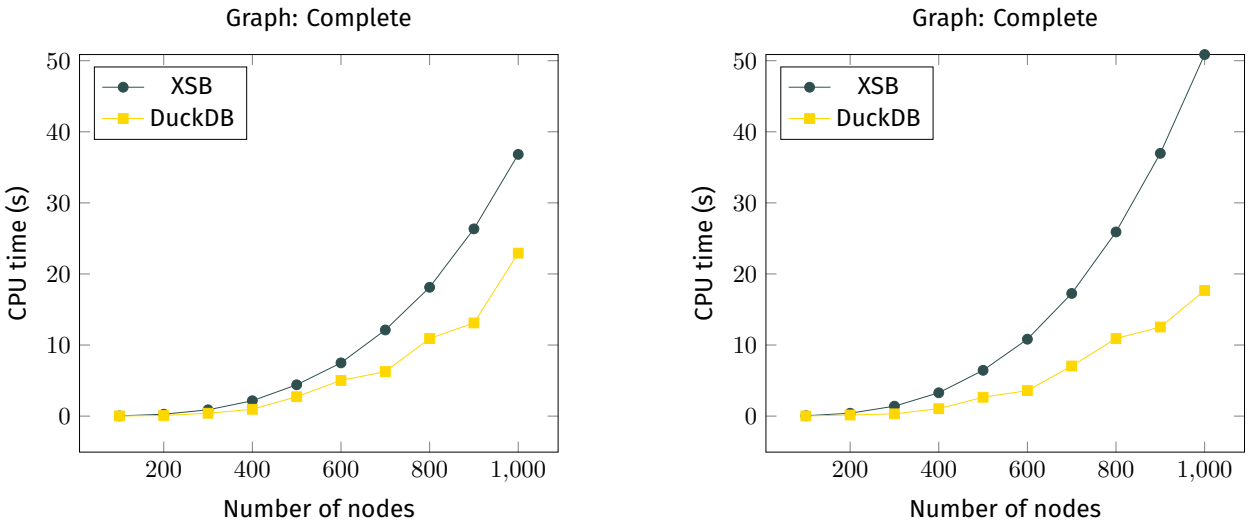
- **Neo4j:** The native graph database was excluded from the final comparative performance analysis due to fundamental architectural limitations for transitive closure computation. First, like MongoDB, Neo4j's Cypher language does not expose tunable recursion semantics. Variable-length path patterns (e.g., `[:EDGE*]`) compute results using a built-in graph traversal algorithm that does not distinguish between left, right, or double recursion [Neo25b], eliminating comparability with systems where recursion strategy is a performance factor.

Second, Neo4j's path enumeration approach is fundamentally unsuited for transitive closure. As documented by Brass and Stephan [BW19], Cypher's variable-length patterns enumerate all paths between nodes before applying `DISTINCT` to deduplicate results. On dense or cyclic graphs, this results in exponential path enumeration to produce a polynomial-sized result. Prior work [BW19] confirmed this yields non-competitive performance compared to declarative recursive queries in logic programming or SQL systems, making Neo4j architecturally inappropriate for this benchmark.

Furthermore, non-linear (double) recursion was excluded from the comparative analysis due to a lack of broad support. Standard-compliant SQL databases like PostgreSQL and CockroachDB forbid it [Pos25, Coc25a], and while DuckDB supports the syntax, it produced incomplete results for 7 of the 12 graph types tested [Ido24]. MariaDB's performance in this recursion strategy was abysmal as well.

4.3 Performance Analysis Across Graph Topologies

The performance of recursive query evaluation is not monolithic; it is highly dependent on the interplay between a database's execution engine and the structural properties of the input graph. Our analysis reveals distinct performance patterns when systems are subjected to graphs of varying density, path length, etc.

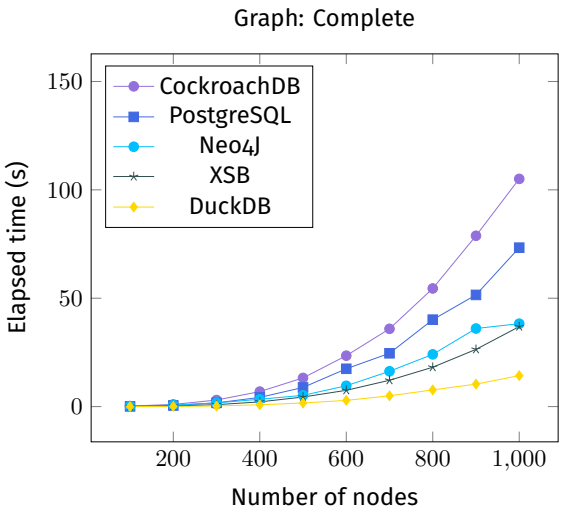


(a) Left Recursion

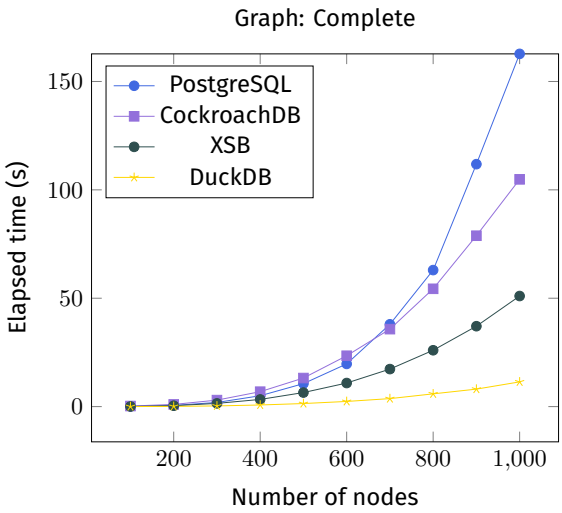
(b) Right Recursion

Figure 1: CPU times on $\texttt{Cmpl}_n$

.



(a) Left Recursion



(b) Right Recursion

MariaDB (Left Recursion)		MariaDB (Right Recursion)	
# nodes	time (s)	# nodes	time (s)
100	0.169	100	0.175
200	1.075	200	1.172
300	3.790	300	4.113
400	9.540	400	10.370
500	19.823	500	21.173
600	33.208	600	35.542
700	> 30 min	700	> 30 min

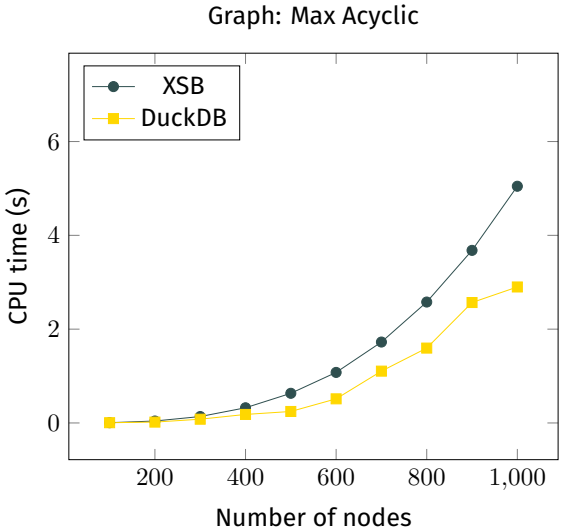
(c) MariaDB is not in the curve because it became drastically slower.

MongoDB	
# nodes	time (s)
100	284.11
200	>10,000 (>150 min)

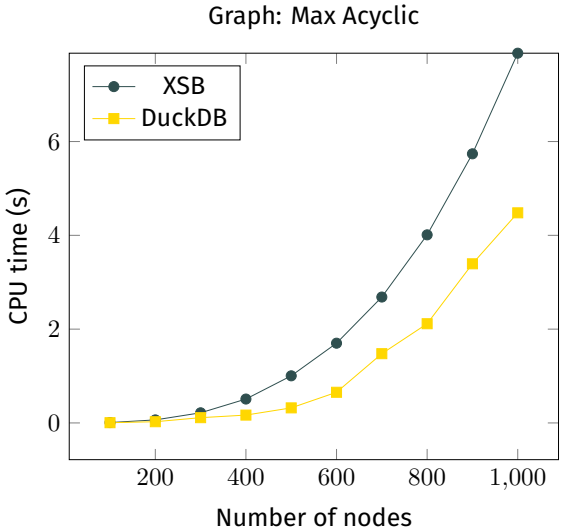
(d) MongoDB is off the charts.

Figure 2: Elapsed times on $\texttt{Cmpl}_n$

.



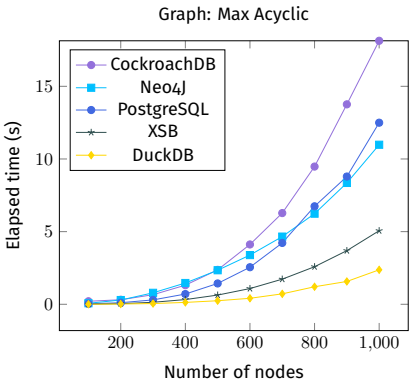
(a) Left Recursion



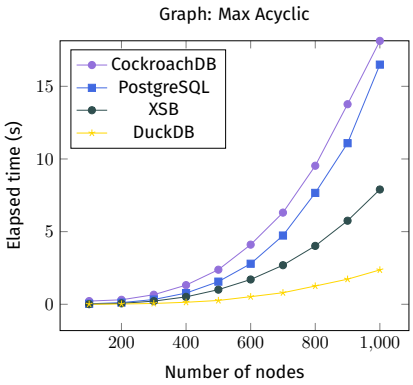
(b) Right Recursion

Figure 3: CPU times on $\texttt{MaxAcyc}_n$

.



(a) Left Recursion



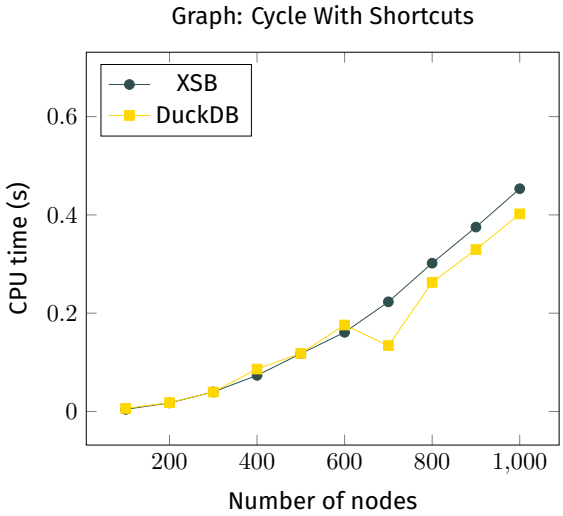
(b) Right Recursion

MariaDB (Left Recursion)		MariaDB (Right Recursion)	
# nodes	time (s)	# nodes	time (s)
100	0.071	100	0.072
200	0.245	200	0.264
300	0.711	300	0.773
400	1.616	400	1.770
500	3.076	500	3.385
600	5.253	600	5.744
700	8.743	700	9.411
800	12.832	800	13.812
900	17.603	900	19.155
1000	> 30 min	1000	> 30 min

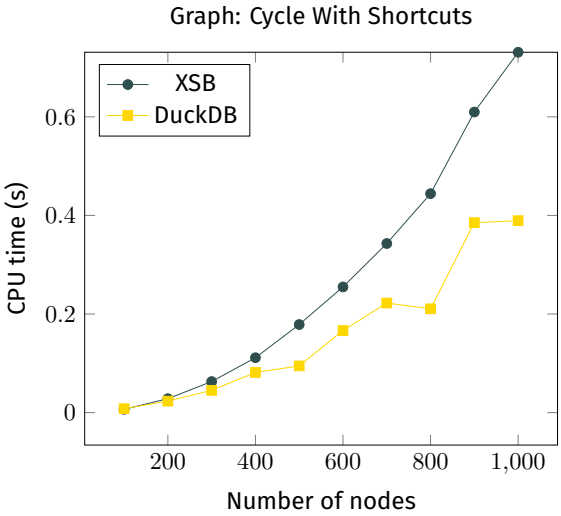
(c) MariaDB is not in the curve because it became drastically slower.

Figure 4: Elapsed time on $\texttt{MaxAcyc}_n$

.



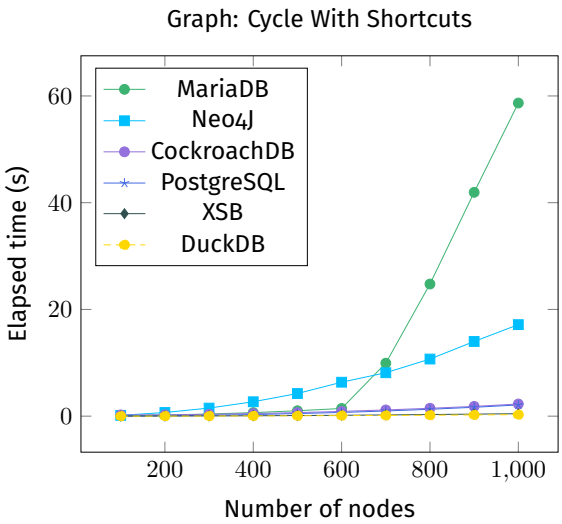
(a) Left Recursion



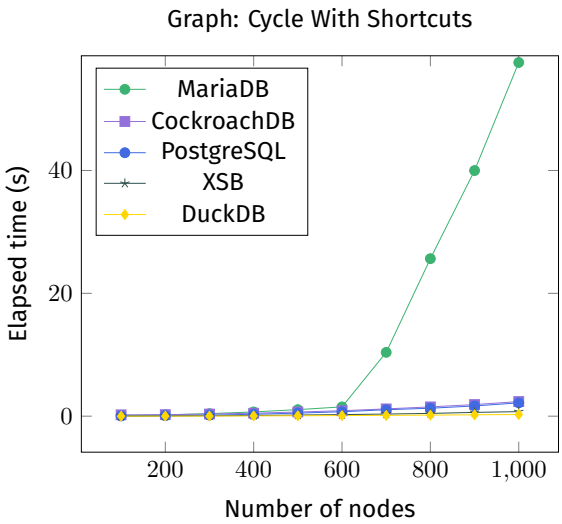
(b) Right Recursion

Figure 5: CPU time on $\texttt{CycExtra}_{n,k}$

.



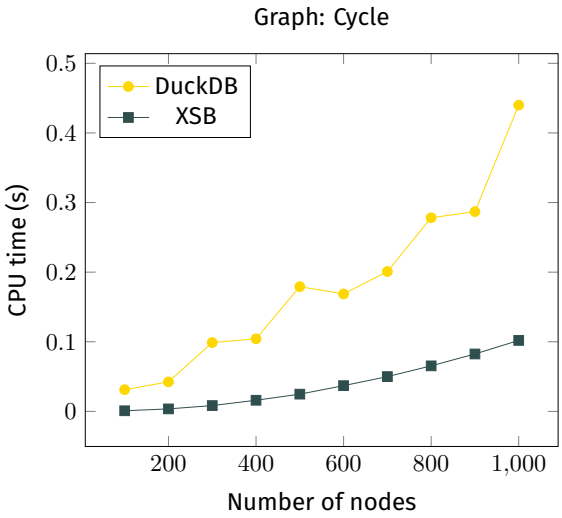
(a) Left Recursion



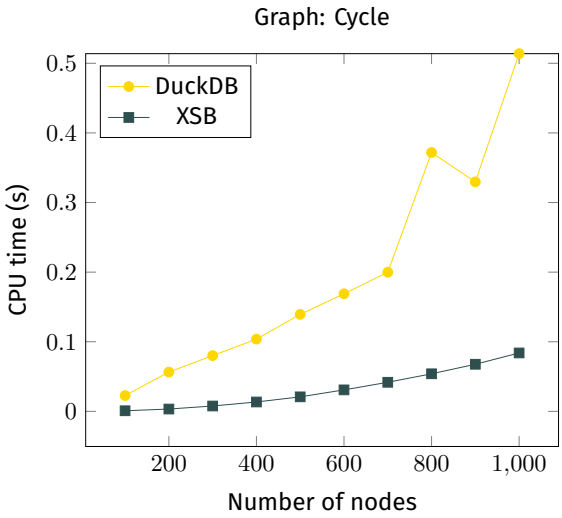
(b) Right Recursion

Figure 6: Elapsed time on $\texttt{CycExtra}_{n,k}$

.



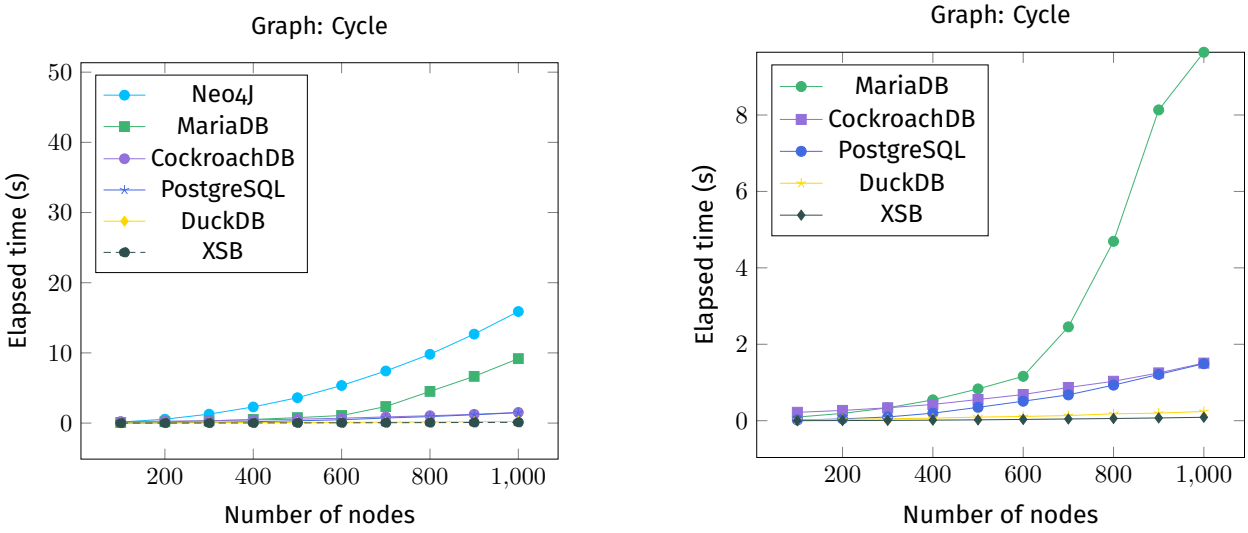
(a) Left Recursion



(b) Right Recursion

Figure 7: CPU time on $\texttt{Cyc}_n$

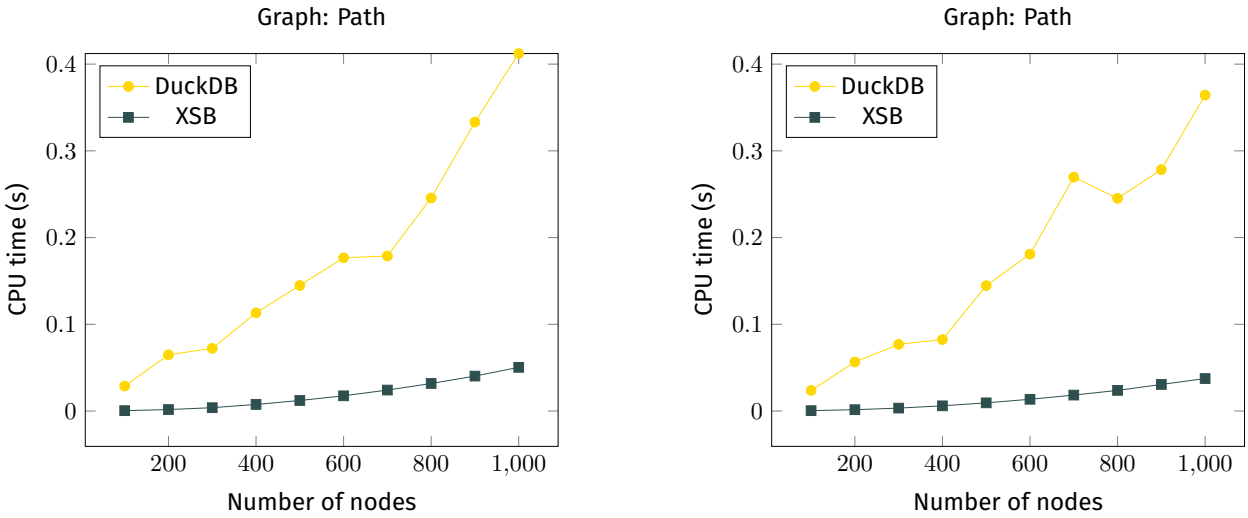
.



(a) Left Recursion (b) Right Recursion

Figure 8: Elapsed time on $\texttt{Cyc}_n$

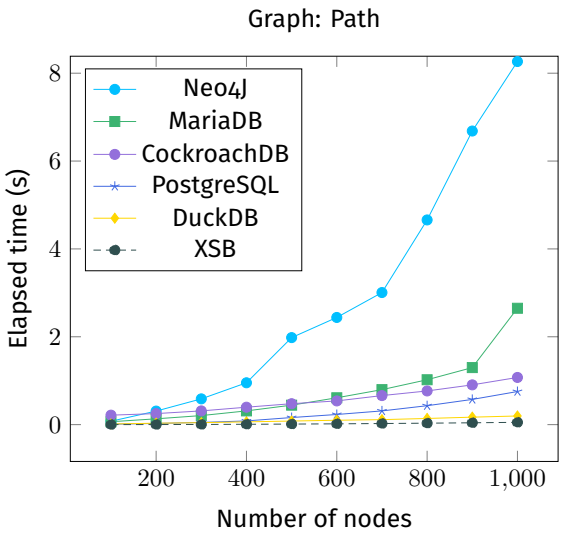
.



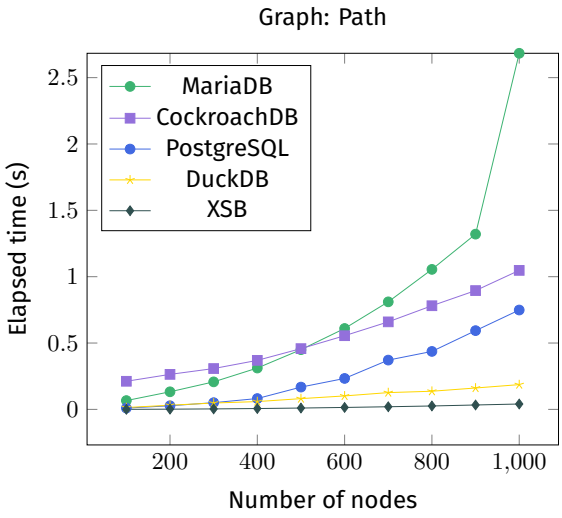
(a) Left Recursion (b) Right Recursion

Figure 9: CPU time on $\texttt{Path}_n$

.



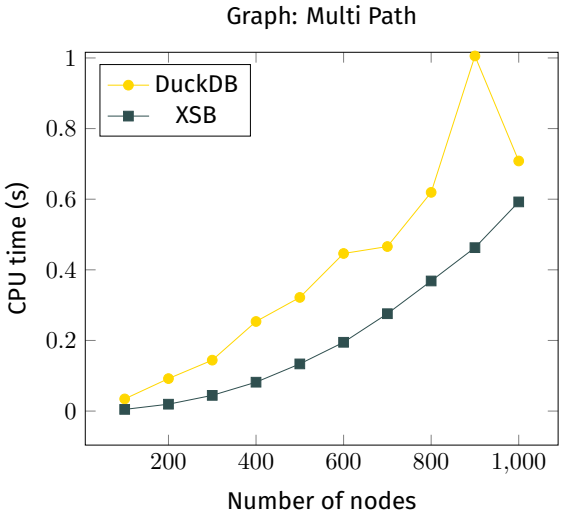
(a) Left Recursion



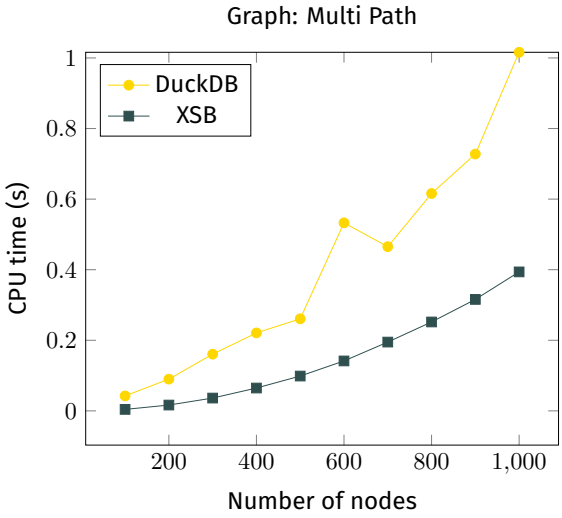
(b) Right Recursion

Figure 10: Elapsed time on $\texttt{Path}_n$

.



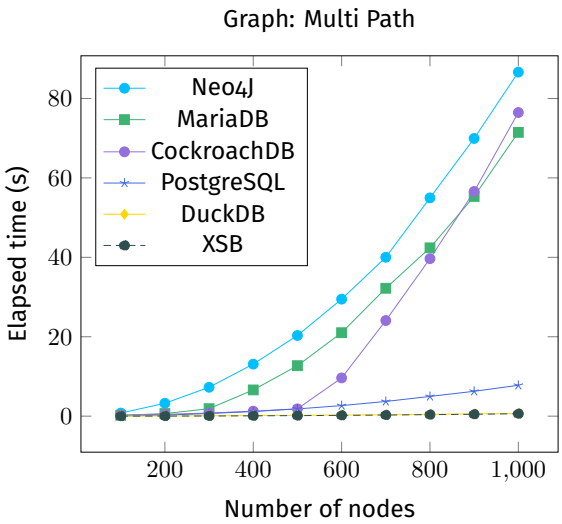
(a) Left Recursion



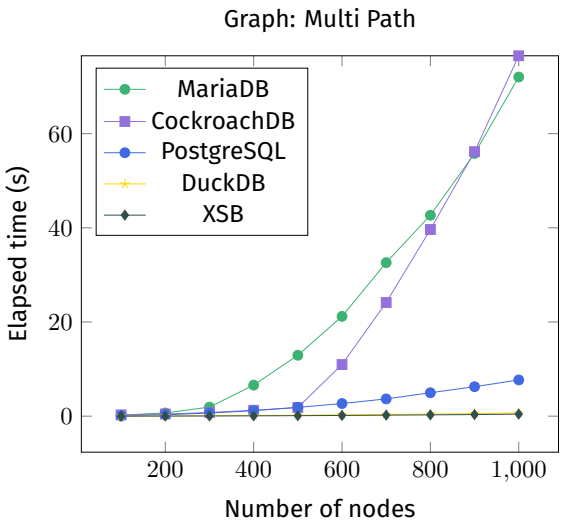
(b) Right Recursion

Figure 11: CPU time on $\texttt{PathDisj}_{n,k=10}$

.



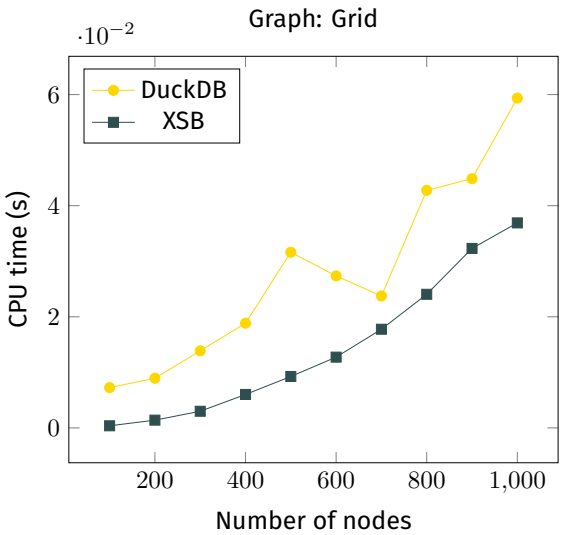
(a) Left Recursion



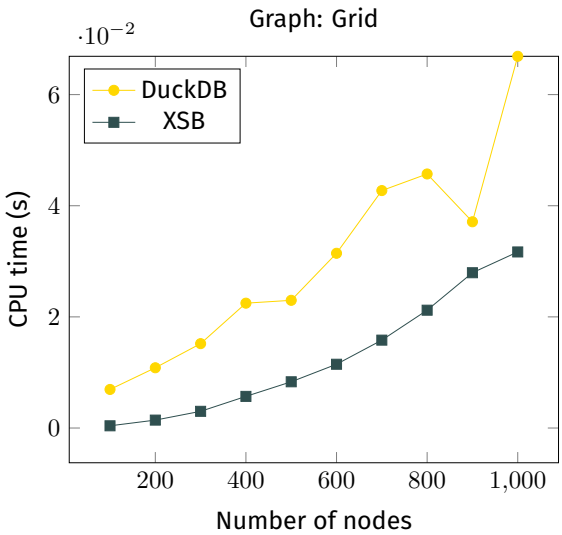
(b) Right Recursion

Figure 12: Elapsed time on $\texttt{PathDisj}_{n,k=10}$

.



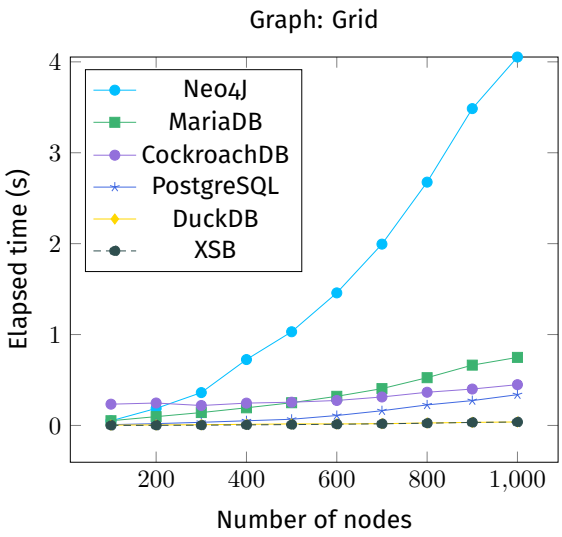
(a) Left Recursion



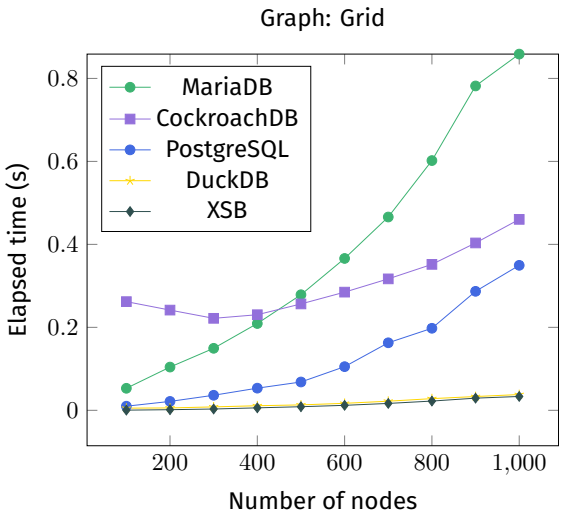
(b) Right Recursion

Figure 13: CPU time on $\texttt{Grid}_n$

.



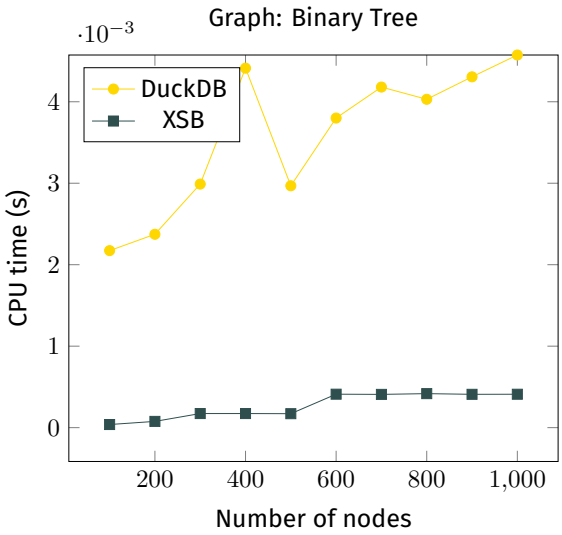
(a) Left Recursion



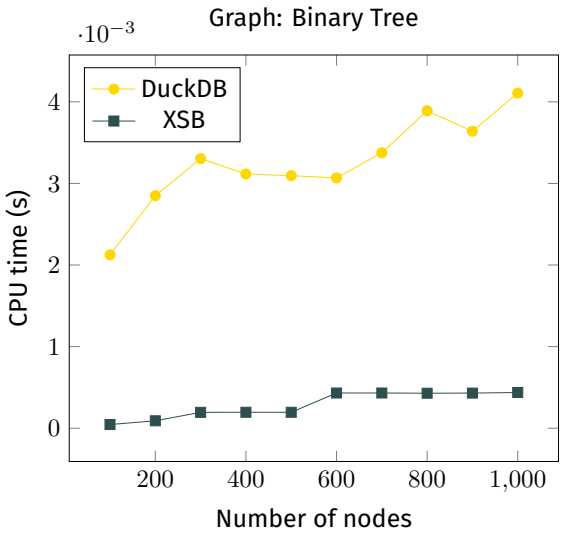
(b) Right Recursion

Figure 14: Elapsed time on $\texttt{Grid}_n$

.



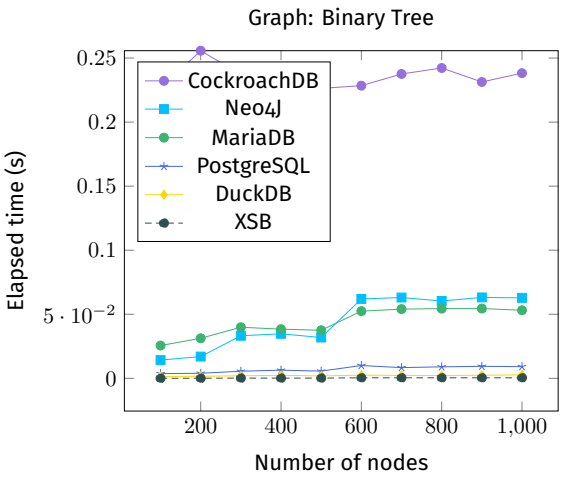
(a) Left Recursion



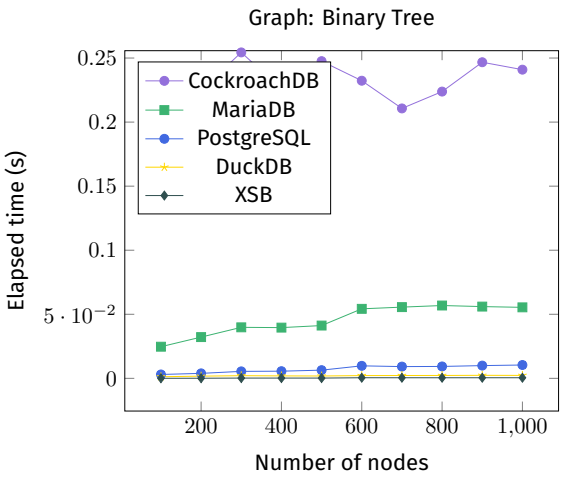
(b) Right Recursion

Figure 15: CPU time on $\texttt{BinTree}_{h=\log_2 n}$

.



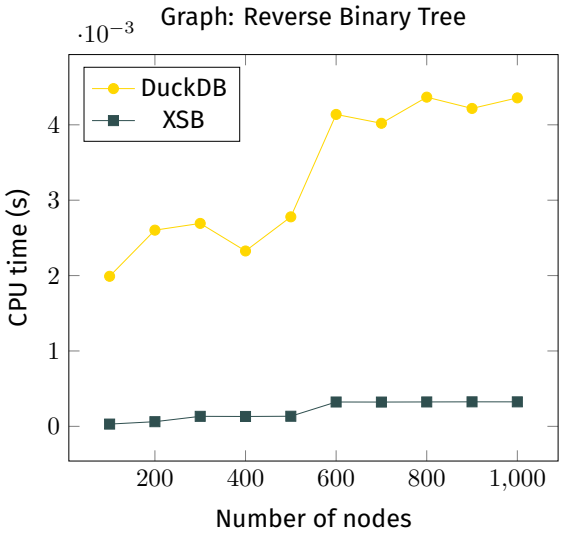
(a) Left Recursion



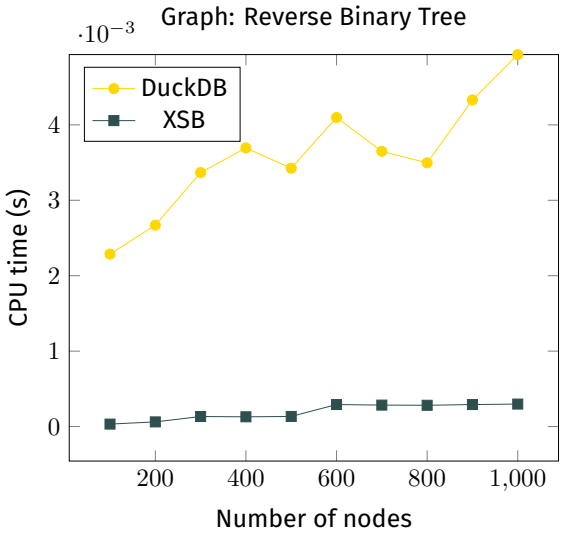
(b) Right Recursion

Figure 16: Elapsed time on $\texttt{BinTree}_{h=\log_2 n}$

.



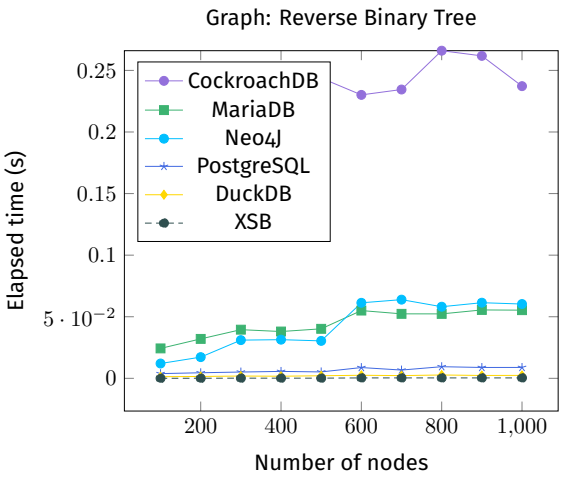
(a) Left Recursion



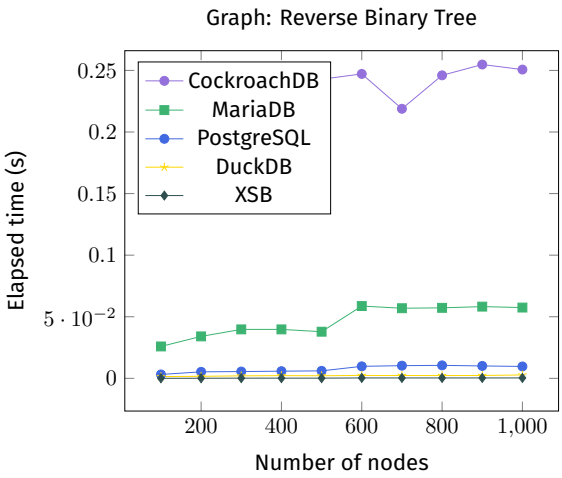
(b) Right Recursion

Figure 17: CPU time on $\texttt{BinTreeRev}_{h=\log_2 n}$

.



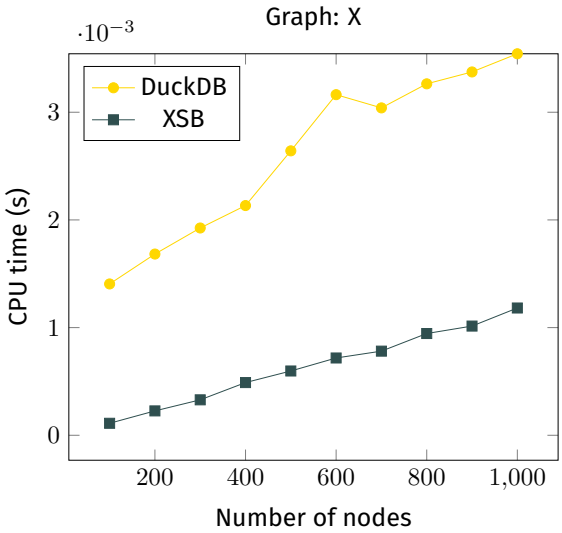
(a) Left Recursion



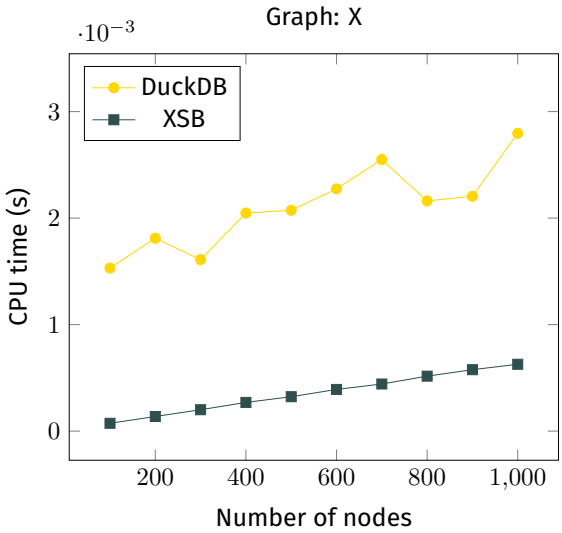
(b) Right Recursion

Figure 18: Elapsed time on $\texttt{BinTreeRev}_{h=\log_2 n}$

.



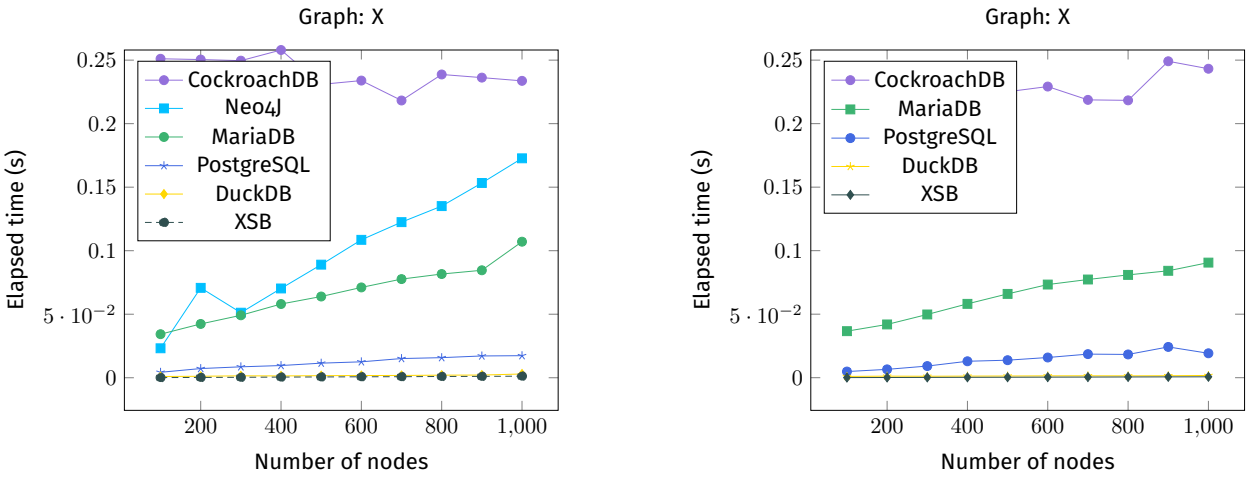
(a) Left Recursion



(b) Right Recursion

Figure 19: CPU time on $\texttt{X}_{n,k=10}$

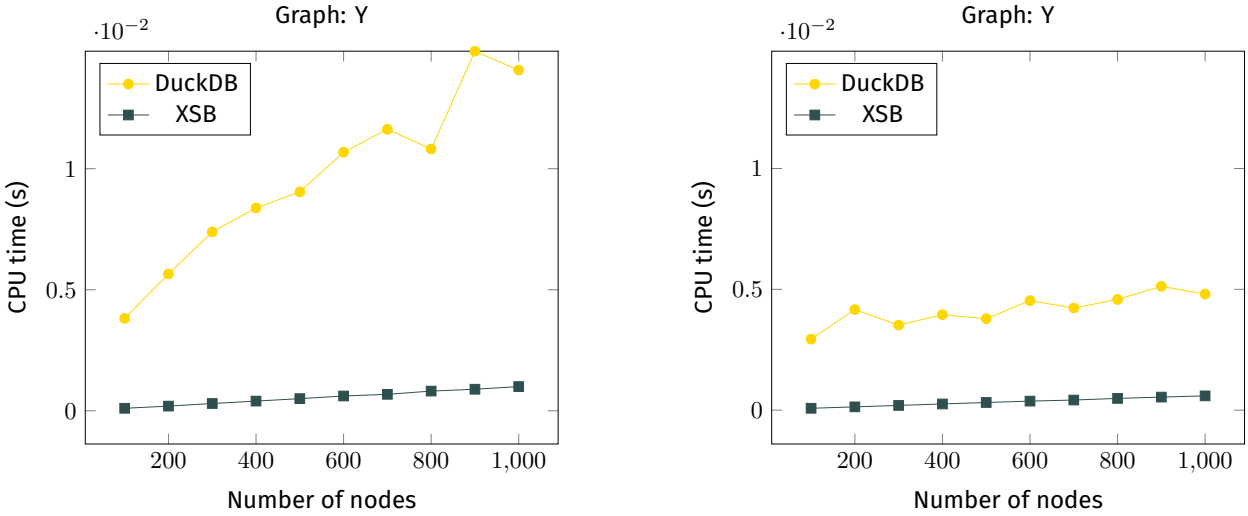
.



(a) Left Recursion (b) Right Recursion

Figure 20: Elapsed time on $\mathtt{X}_{n,k=10}$

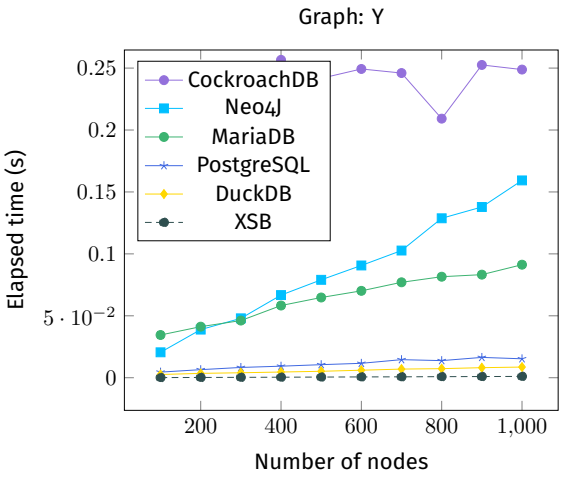
.



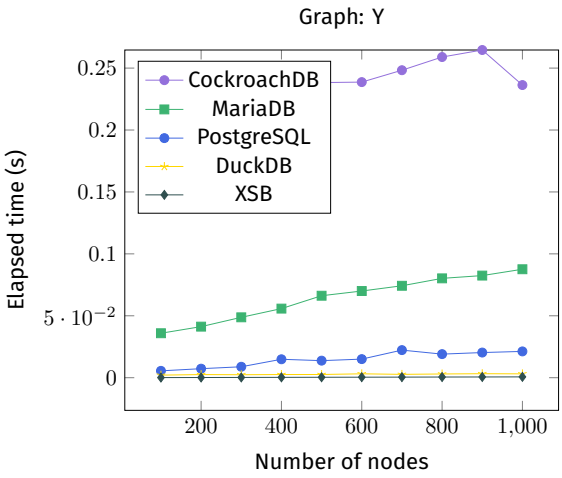
(a) Left Recursion (b) Right Recursion

Figure 21: CPU time on $\mathtt{Y}_{n,k=10}$

.



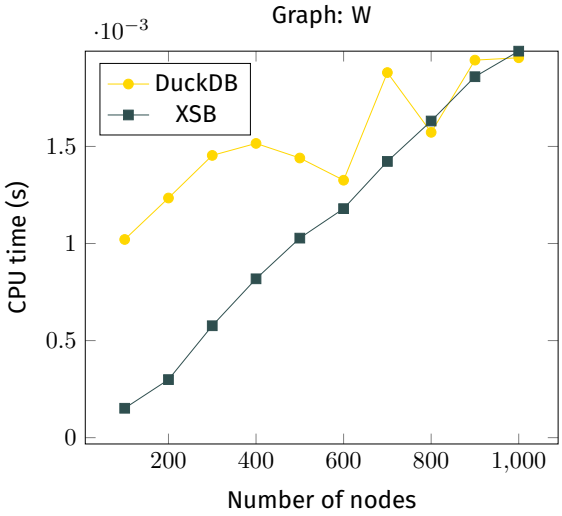
(a) Left Recursion



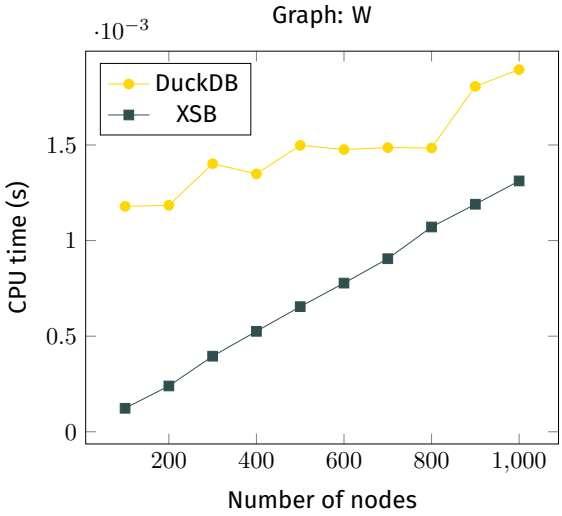
(b) Right Recursion

Figure 22: Elapsed time on $\mathtt{Y}_{n,k=10}$

.



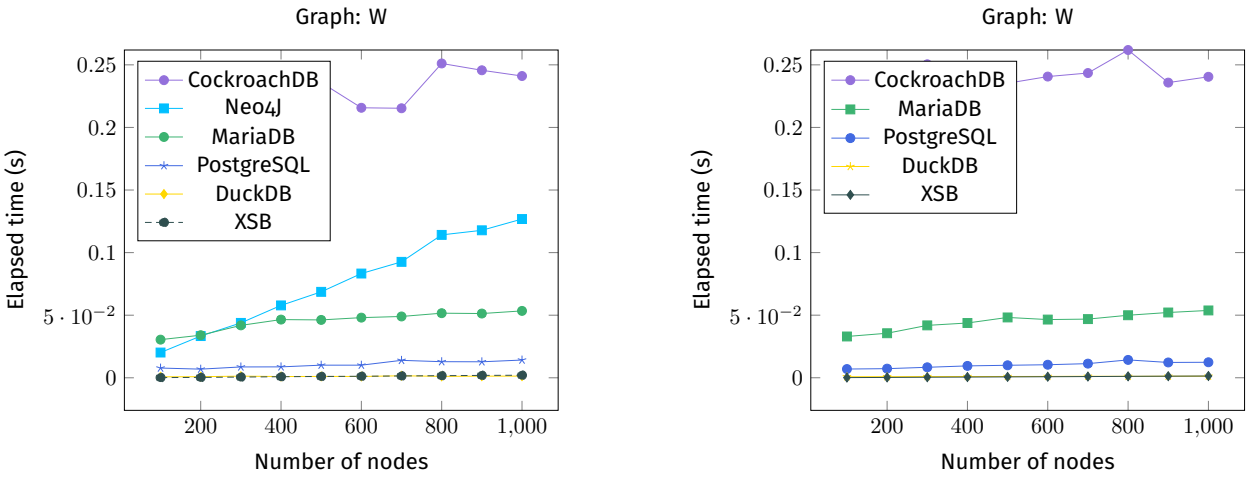
(a) Left Recursion



(b) Right Recursion

Figure 23: CPU time on $\mathtt{W}_{n,k=10}$

.



(a) Left Recursion

(b) Right Recursion

Figure 24: Elapsed time on $\mathtt{W}_{n,k=10}$

.

Dense Graphs: The Impact of cubic Combinations Dense graphs, complete graphs ($Cmpl_n$) and maximum acyclic graphs ($MaxAcyc_n$) in particular, make a system work hard. A system needs to consider at least a cubic number of combinations of edges and paths in number of vertices.

As illustrated in Figures 2 and 4, DuckDB consistently demonstrates superior performance on these workloads. This is a direct consequence of its vectorized, columnar execution engine [RM19b]. By processing data in large batches (vectors) rather than tuple-by-tuple, vectorized execution drastically reduces per-row interpretation overhead and maximizes CPU cache efficiency, a model pioneered by systems like MonetDB/X100 [BZN05]. This architecture is exceptionally well-suited for the large, uniform join operations that dominate each iteration of transitive closure on dense graphs.

In contrast, the traditional row-oriented engines of PostgreSQL and CockroachDB, while robust, are less efficient under these conditions. Their performance, though respectable, is an order of magnitude slower than DuckDB's. MariaDB exhibits a catastrophic performance degradation, failing to complete queries on larger dense graphs within the allotted time. This suggests that its query planner and the underlying storage engine are not optimized for the massive intermediate temporary tables generated during these specific recursive computations.

The logic programming system XSB shows competitive performance, outperforming the row-oriented RDBMSs but lagging behind DuckDB. XSB's efficiency stems from its use of tabling, a sophisticated form of memoization that avoids re-computation of identical subgoals [SW12, War92]. However, the raw computational throughput of DuckDB's vectorized joins proves superior to the more complex goal-driven evaluation of XSB.

Sparse Graphs: A linear factor fewer combinations Sparse graphs with long path lengths, paths ($Path_n$) and cycles (Cyc_n) in particular, test a different aspect of performance: the overhead of the recursive iteration itself. In these cases, the number of iterations is still high (proportional to n), but the number of new tuples generated per iteration is a linear factor smaller.

On these workloads (Figures 10 and 7), the best performed system is no longer DuckDB. XSB consistently outperforms all other systems. Its goal-directed, top-down evaluation strategy with tabling is exceptionally efficient, as it only explores relevant paths [SW12, War92].

Conversely, this is where the architectural trade-offs of DuckDB's vectorized engine become apparent. As acknowledged by its developers, when each iteration produces only a few rows, the overhead of managing and processing vectors can outweigh the benefits, leading to performance that is slower than both XSB and, in some cases, the traditional RDBMSs [Git22]. The performance of PostgreSQL and CockroachDB remains stable, demonstrating the maturity of their query execution engines for general-purpose workloads.

Workload Type	**System Recommendation**	**Architectural Strength / Evaluation Method**	**Most Efficient Recursion**
Dense Graph Analytics	DuckDB	Vectorized Columnar Engine, high-throughput joins	Right
Deep, Sparse Traversals	XSB	Tabled Logic Programming, query-driven evaluation	Left
General-Purpose / Mixed Workloads	PostgreSQL	Mature Row-Store RDBMS, robust query optimizer	Left, Right
Distributed / Fault-Tolerant	CockroachDB	Distributed SQL Architecture (single-node not primary use case)	Left, Right

Table 2: System recommendations and optimal recursion types for different workloads.

The Influence of Join Directionality: Left vs. Right Recursion The choice between left and right recursion, while semantically equivalent for transitive closure, had a measurable impact on performance that varied by system and graph topology. This difference arises from the query optimizer's choice of physical join algorithms (e.g., hash join vs. nested loop join) and its selection of the inner (build) and outer (probe) relations for that join [Gra93].

For instance, on dense graphs like $Cmpl_n$, DuckDB performed better with right recursion, suggesting its optimizer favored using the smaller, static edge table as the build side of a hash join against the larger, growing `tc` relation. In contrast, XSB's performance on the same graph was superior with left recursion. This is because its top-down, goal-directed evaluation is more efficient when the recursive predicate (`tc`) is expanded first, allowing its bindings to constrain the subsequent lookup in the edge table. These subtle differences underscore the fact that even for semantically equivalent queries, performance is deeply tied to the internal execution and optimization strategies of each system.

4.4 Architectural Implications and Overall Findings

Our experimental results lead to a clear conclusion: there is no single "best" system for all recursive workloads. Performance is fundamentally determined by a system's architectural specialization and by workload characteristics. Table 3 summarizes our findings and provides recommendations for practitioners and researchers.

System	Architectural Profile	Recommended Use Case
DuckDB	High-throughput, vectorized, columnar analytical engine.	**Dense Graph Analytics:** Ideal for workloads with large intermediate join results. Its CPU-centric, cache-efficient model provides leading performance for analytical recursion.
PostgreSQL	Mature, general-purpose, row-oriented RDBMS.	**General-Purpose & Standard-Compliant Recursion:** A robust and predictable baseline. Recommended when versatility and strict SQL compliance are required, and performance, while not peak, must be reliable across varied graph types.
XSB	Tabled logic programming system with goal-directed, top-down evaluation.	**Deep, Sparse Traversals:** Unmatched performance on workloads dominated by iteration overhead rather than data volume (e.g., long paths, cycles). Best for tasks where minimizing redundant computation paths is critical.
CockroachDB	Distributed-first, transactional SQL database.	**Geo-Distributed, Fault-Tolerant Workloads:** While single-node performance is impacted by transactional overhead, its architecture is designed for multi-node resilience and scalability (evaluation in a distributed setting is future work).
MongoDB, SingleStore, Neo4j	Document store, distributed SQL with limited CTEs, and native graph database.	**Not Recommended for Transitive Closure Benchmarks:** Excluded due to prohibitive performance scaling ($graphLookup) and functional limitations (crippled CTE support, non-tunable recursion strategy).

Table 3: System Recommendations for Recursive Workloads

DuckDB's Analytics performance DuckDB is the unequivocal performance leader for analytics over dense graphs, where recursive computation involves large intermediate join results. Its vectorized, columnar execution model [RM19b, BZN05] achieves high throughput by optimizing for CPU cache locality and minimizing per-tuple overhead. However, for sparse graphs with long paths, its advantages diminish; overheads inherent to vector management

can lead to suboptimal performance, and support for non-standard recursive features such as non-linear recursion is still incorrect [Ido24, Git22].

PostgreSQL's General-Purpose Robustness PostgreSQL offers robust and predictable performance across diverse graph topologies. As a mature row-store RDBMS, its optimizer produces reasonable plans that avoid catastrophic failures—even when not matching the peak throughput of DuckDB on big-data analytical workloads. Its full SQL:1999 compliance and broad community adoption make it a safe, versatile baseline for standards-compliant recursion [Pos25].

CockroachDB's Distributed Processing Overhead In single-node configuration, CockroachDB typically performs more slowly than PostgreSQL, especially on workloads with low concurrency or small graphs. This is attributable to transactional and distribution overheads, as its architecture is fundamentally designed for geo-distributed, fault-tolerant workloads [Taf20]. This overhead, while justified for high-resilience, multi-node deployments, can be a disadvantage in local analytical scenarios compared to single-node-optimized systems.

XSB's Logic Programming Engine Efficiency The tabled logic programming system XSB outperforms relational solutions on sparse graphs. Its goal-directed evaluation model, supported by efficient tabling [SW12, War92], allows highly targeted search that minimizes redundant computation. However, it cannot exploit set-based join optimizations, limiting performance on dense graphs.

In summary, application architects should select the database platform for recursive workloads by considering both graph structure and query semantics:

- For dense data or analytical tasks, a vectorized engine like DuckDB is ideal.
- For general-purpose, standards-based recursion, PostgreSQL is reliable and robust.
- For deep, sparse traversals, a logic programming engine like XSB outperforms mainstream SQL systems.
- Systems such as MongoDB (with $graphLookup) and SingleStore (with crippled recursion support) are inadequate for serious recursive or transitive closure analytics [Mon25b, Mon25a, Sin25].

Future work should include distributed configurations, alternative logic and Datalog systems, and finer-grained workload-adaptive query optimizations in both relational and analytical engines.

5 Related work

There has been extensive study of recursive query processing efficiency in database systems, especially through algorithmic analysis and experimental evaluation. The initial motivation stemmed from the expressiveness limitations of relational algebra, which cannot compute the transitive closure of a relation [AU79]. This led to the introduction of Datalog as a declarative language for expressing recursive queries over relational databases [Ull88, CGT90].

Early work focused on establishing fundamental evaluation algorithms. The Naive Evaluation method provided a simple, iterative fixpoint computation but was highly inefficient due to redundant work in each iteration [Ban86]. This was quickly superseded by Semi-Naive Evaluation, which optimizes the process by only considering new facts generated in the previous iteration, drastically reducing redundant computations and becoming the baseline for most bottom-up implementations [AK19, BR87]. These foundational algorithms were primarily analyzed theoretically, establishing their soundness, completeness, and basic complexity bounds.

A major thrust in optimization research was to avoid computing irrelevant facts for a given query. The Magic Sets transformation is the seminal work in this area, rewriting a Datalog program to simulate a top-down, goal-directed evaluation strategy within a bottom-up framework, often yielding orders-of-magnitude performance gains for queries with bound arguments [BMSU85, MFPR90, BR91]. Alternative top-down approaches with memoization (or tabling) were also explored, particularly in the logic programming community, which influenced systems like XSB [War92, SW12] and demonstrated strong performance for similar query patterns. Recent advancements have provided a more formal basis for comparing these goal-directed strategies. The work by Tekle and Liu, for instance, offers precise time and space complexity analyses for top-down goal-directed Datalog evaluation, establishing a clear efficient hierarchy [Tek10, TL10]. Their findings demonstrate that top-down evaluation with subsumptive tabling, which allows for greater reuse of computed answers, is provably more efficient than standard (variant) tabling, which in turn is at least as efficient as the Magic Sets transformation [TL11]. Other optimization techniques include counting methods and specialized transformations [Nau89, SZ86, CGV03].

The integration of recursion into mainstream SQL via the WITH RECURSIVE common table expression (CTE) shifted focus to system implementation challenges within existing query optimizers and execution engines [SHP+96]. Studies examined the difficulties of cost estimation for recursive queries, as the number of iterations is data-dependent, making it difficult for traditional cost-based optimizers to choose optimal plans [PHH92]. The performance of recursive CTEs was shown to be fundamentally tied to the efficiency of the underlying join and union or union all operations, with hash-based algorithms and efficient duplicate elimination being critical for scalability [Gra93].

Comparative studies and benchmarks have been developed to evaluate system performance. While some works compare specific Datalog engines [JSS16, KKNV23, KK22], others compare recursion support across different relational systems [BW19]. More recent efforts like the LDBC benchmark focus on graph queries, which heavily involve recursion, but are primarily targeted at specialized graph databases [ABLP+14]. Comprehensive comparisons

across architectural paradigms—Relational, NoSQL, and NewSQL—on fundamental recursion types are less common. Some works have compared relational systems against graph databases like Neo4j, typically finding that the native traversal approach of graph databases using index-free adjacency offers superior performance for deep path queries compared to the iterative join-based approach of RDBMS [CAH12, VMZ+10b].

The performance characteristics of modern database architectures for recursion are a recent focus. The rise of columnar, vectorized execution engines as seen in DuckDB has shown to dramatically speed up the scan-and-join loops inherent in recursive CTEs, leveraging CPU caches more effectively than traditional row-oriented systems [BZN05, ZHNB07]. In contrast, distributed SQL (NewSQL) systems face significant challenges due to the high communication overhead of shuffling intermediate results between nodes at each iteration, a problem theoretically analyzed in MapReduce-like models [ABC+11]. This often leads such systems to restrict support to linear recursion or exhibit poor scalability for complex recursive workloads. NoSQL document stores like MongoDB offer limited, often inefficient, support through specific operators like $graphLookup, which cannot be generalized and are not competitive with native implementations [Mon25b].

Unlike our work, few of these studies perform a systematic analysis and prediction of performance across all three recursion types (left, right, and non-linear/double) on a broad suite of graph types. Furthermore, none conduct a direct, empirical comparison of leading systems from each of the dominant modern architectural classes (SQL, NoSQL, NewSQL) alongside a highly optimized logic programming system. Additionally, our investigation into the correctness of results for non-linear recursion, as seen with DuckDB, addresses a critical but often overlooked aspect of system evaluation [Ido24].

The theoretical complexity studies of bottom-up and top-down evaluation have been essential for our work [Ull90, TL11]. These formal analyses provide the basis for understanding the performance characteristics of different recursion strategies and for predicting how different graph structures will impact execution time. For instance, the distinction between linear and non-linear recursion has profound complexity implications [CH82], which directly explains the implementation choices and performance limitations observed in systems like PostgreSQL and CockroachDB.

5.1 Conclusion, limitations, and future works

In this work, we present a systematic experimental study and performance prediction of recursive query processing across a diverse set of modern database systems. Despite providing compelling insights for practitioners choosing a system and for researchers building new ones, this work has several limitations and opens many avenues for future work.

First, our analysis focuses primarily on elapsed execution time. A more comprehensive study would also measure and analyze resource utilization in detail, including CPU usage, memory consumption for intermediate results, and I/O patterns [KK22], especially as data size exceeds main memory. Second, our study uses a single-node setup for all systems. A crucial area for future work is to evaluate the scalability and performance of the distributed systems (CockroachDB, SingleStore) in a multi-node cluster, which would expose the net-

work communication costs that are central to their performance profile. Third, we evaluate the systems' default configurations. An in-depth study of system-specific tuning parameters, indexing strategies (e.g., GIN vs. B-Tree in PostgreSQL), and physical data layout could reveal further performance optimizations. Finally, while we test three fundamental recursion patterns, many other recursive algorithms could be benchmarked, including those involving aggregation, which are critical for many graph analytics problems like shortest path calculations with weighted edges.

References

[AAB+23] Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan Reutter, and Domagoj Vrgoc. Demystifying graph databases: Analysis and taxonomy of data organization, system designs, and graph queries. *ACM Computing Surveys*, 56(2):1–40, 2023.

[ABC+11] Foto N. Afrati, Vinayak Borkar, Michael Carey, Neoklis Polyzotis, and Jeffrey D. Ullman. Map-reduce extensions and recursive queries. In *Proceedings of the 14th International Conference on Extending Database Technology*, EDBT/ICDT '11, page 1–8, New York, NY, USA, 2011. Association for Computing Machinery.

[ABD+16] Dakshi Agrawal, Ali Butt, Kshitij Doshi, Josep-L Larriba-Pey, Min Li, Frederick R Reiss, Francois Raab, Berni Schiefer, Toyotaro Suzumura, and Yinglong Xia. Sparkbench–a spark performance testing suite. In *Performance Evaluation and Benchmarking: Traditional to Big Data to Internet of Things: 7th TPC Technology Conference, TPCTC 2015, Kohala Coast, HI, USA, August 31–September 4, 2015. Revised Selected Papers 7*, pages 26–44. Springer, 2016.

[ABLP+14] Renzo Angles, Peter Boncz, Josep Larriba-Pey, Irini Fundulaki, Thomas Neumann, Orri Erling, Peter Neubauer, Norbert Martinez-Bazan, Venelin Kotsev, and Ioan Toma. The linked data benchmark council: a graph and rdf industry benchmarking effort. *SIGMOD Rec.*, 43(1):27–31, May 2014.

[AFB15] Rafael Almeida, Pedro Furtado, and Jorge Bernardino. Performance evaluation mysql innodb and microsoft sql server 2012 for decision support environments. In *Proceedings of the Eighth International C* Conference on Computer Science & Software Engineering*, C3S2E '15, page 56–62, New York, NY, USA, 2015. Association for Computing Machinery.

[AK19] Michael Arntzenius and Neel Krishnaswami. Seminaïve evaluation for a higher-order functional language. *Proc. ACM Program. Lang.*, 4(POPL), December 2019.

[AU79] Alfred V. Aho and Jeffrey D. Ullman. Universality of data retrieval languages. In *Proceedings of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of*

Programming Languages, POPL '79, page 110–119, New York, NY, USA, 1979. Association for Computing Machinery.

[Ban86] Francois Bancilhon. *Naive evaluation of recursively defined relations*, page 165–178. Springer-Verlag, Berlin, Heidelberg, 1986.

[BB16] Narsimhulu Banothu and Satish Narayana Bhukya. Big-data: Acid versus base for database transactions. In *2016 IEEE Region 10 Conference (TENCON)*, pages 3572–3575. IEEE, 2016.

[BDT83] Dina Bitton, David J. DeWitt, and Carolyn Turbyfill. Benchmarking database systems a systematic approach. In *Very Large Data Bases Conference*, 1983.

[BEA+20] Leilani Battlde, Philipp Eichmann, Marco Angelini, Tiziana Catarci, Giuseppe Santucci, Yukun Zheng, Carsten Binnig, Jean-Daniel Fekete, and Dominik Moritz. Database benchmarking for supporting real-time interactive querying of large data. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, SIGMOD '20, page 1571–1587, New York, NY, USA, 2020. Association for Computing Machinery.

[BMPB21] Tiemo Bang, Norman May, Ilia Petrov, and Carsten Binnig. Anydb: An architecture-less dbms for any workload. In *Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR)*, 2021.

[BMSU85] Francois Bancilhon, David Maier, Yehoshua Sagiv, and Jeffrey D Ullman. Magic sets and other strange ways to implement logic programs (extended abstract). In *Proceedings of the Fifth ACM SIGACT-SIGMOD Symposium on Principles of Database Systems*, PODS '86, page 1–15, New York, NY, USA, 1985. Association for Computing Machinery.

[Boe20] Matthias Boehm. Technical perspective: Declarative recursive computation on an rdbms. *SIGMOD Record*, 49(3):47–48, 2020.

[BR87] I. Balbin and K. Ramamohanarao. A generalization of the differential approach to recursive query evaluation. *J. Log. Program.*, 4(3):259–262, September 1987.

[BR91] Catriel Beeri and Raghu Ramakrishnan. On the power of magic. *J. Log. Program.*, 10(3–4):255–299, March 1991.

[BW19] Stefan Brass and Mario Wenzel. Performance analysis and comparison of deductive systems and SQL databases. In Mario Alviano and Andreas Pieris, editors, *Datalog 2.0 2019 - 3rd International Workshop on the Resurgence of Datalog in Academia and Industry co-located with the 15th International Conference on Logic Programming and Nonmonotonic Reasoning (LPNMR 2019) at the Philadelphia Logic Week 2019, Philadelphia, PA (USA), June 4-5, 2019*, volume 2368 of *CEUR Workshop Proceedings*, pages 27–38. CEUR-WS.org, 2019.

[BWS14] Aleksandra Boniewicz, Piotr Wiśniewski, and Krzysztof Stencel. On materializing paths for faster recursive querying. In *New Trends in Databases and Information Systems: 17th East European Conference on Advances in Databases and Information Systems*, pages 105–112. Springer, 2014.

[BZN05] Peter A. Boncz, Marcin Zukowski, and Niels Nes. Monetdb/x100: Hyper-pipelining query execution. In *Second Biennial Conference on Innovative Data Systems Research, CIDR 2005, Asilomar, CA, USA, January 4-7, 2005, Online Proceedings*, pages 225–237. www.cidrdb.org, 2005.

[CAH12] Marek Ciglan, Alex Averbuch, and Ladialav Hluchy. Benchmarking traversal operations over graph databases. In *Proceedings of the 2012 IEEE 28th International Conference on Data Engineering Workshops*, ICDEW '12, page 186–189, USA, 2012. IEEE Computer Society.

[CGT90] Stefano Ceri, Georg Gottlob, and Letizia Tanca. *Logic programming and databases*. Springer-Verlag, Berlin, Heidelberg, 1990.

[CGV03] Diego Calvanese, Giuseppe De Giacomo, and Moshe Y. Vardi. Decidable containment of recursive queries. In *Proceedings of the 9th International Conference on Database Theory*, ICDT '03, page 330–345, Berlin, Heidelberg, 2003. Springer-Verlag.

[CH82] Ashok K. Chandra and David Harel. Horn clauses and the fixpoint query hierarchy. In *Proceedings of the 1st ACM SIGACT-SIGMOD Symposium on Principles of Database Systems*, PODS '82, page 158–163, New York, NY, USA, 1982. Association for Computing Machinery.

[Coc25a] Cockroach Labs. Common table expressions (with queries). `https://www.cockroachlabs.com/docs/stable/common-table-expressions`, 2025.

[Coc25b] Cockroach Labs. Known limitations. `https://www.cockroachlabs.com/docs/stable/known-limitations.html`, 2025.

[CTW+21] Zhuhua Chen, Jie Tan, Jingren Wang, Gang Chen, Feifei Wang, and Jianhua Feng. Singlestore: A real-time distributed sql database. *Proceedings of the VLDB Endowment*, 14(12):3245–3258, 2021.

[Dah17] Kristofer Dahlgren. Dial m for management: Next generation nosql. In *Proceedings of the 8th Conference on Innovative Data Systems Research (CIDR)*, 2017.

[DCH05] David J DeWitt, Michael J Carey, and Joseph M Hellerstein. Stonebraker receives ieee john von neumann medal. *SIGMOD Record*, 34(2):5–6, 2005.

[DG24] Federico Di Gregorio. psycopg2 - python-postgresql database adapter. `https://pypi.org/project/psycopg2/`, 2024.

[Duc24a] DuckDB. Duckdb python api. `https://duckdb.org/docs/api/python/overview`, 2024.

[Duc24b] DuckDB Labs. *Recursive CTEs*, 2024.

[EM99] Andrew Eisenberg and Jim Melton. Sql: 1999, formerly known as sql3. *SIGMOD Rec.*, 28(1):131–138, mar 1999.

[FGS+21] Stelios Floratos, Ahmad Ghazal, Jie Yu Sun, Jianqiu Chen, and Xiaolei Zhang. Dbspinner: Making a case for iterative processing in databases. In *Proceedings of the 37th IEEE International Conference on Data Engineering (ICDE)*, pages 2304–2315. IEEE, 2021.

[FH17] Marin FOTACHE and Ionuț HRUBARU. Performance analysis of two big data technologies on a cloud distributed architecture. results for non-aggregate queries on medium-sized data. *Scientific Annals of Economics and Business*, 63(SI):21–50, Jan. 2017.

[Git22] GitHub. Recursive cte with 1 row per iteration is very slow. `https://github.com/duckdb/duckdb/issues/8700`, 2022.

[Gra92] Jim Gray. *Benchmark Handbook: For Database and Transaction Processing Systems*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1992.

[Gra93] Goetz Graefe. Query evaluation techniques for large databases. *ACM Comput. Surv.*, 25(2):73–169, June 1993.

[GV20] Zvonimir Galić and Marko Vuzem. A generic and extensible core and prototype of consistent, distributed, and resilient lis. *ISPRS International Journal of Geo-Information*, 9(7):437, 2020.

[GWD21] Brink Glasbergen, Fei Wu, and Khuzaima Daudjee. Dendrite: Bolt-on adaptivity for data systems. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, pages 649–661. ACM, 2021.

[HR83] Theo Härder and Andreas Reuter. Principles of transaction-oriented database recovery. *ACM Computing Surveys*, 15(4):287–317, 1983.

[Ido24] John Idogun. Inconsistent results with recursive queries on some graphs in duckdb, 2024. Accessed: 2024-08-01.

[Inc24] Gartner Inc. Magic quadrant for cloud database management systems, 2024.

[JSS16] Herbert Jordan, Bernhard Scholz, and Pavle Subotic. Soufflé: On synthesis of program analyzers. In Swarat Chaudhuri and Azadeh Farzan, editors, *Computer Aided Verification - 28th International Conference, CAV 2016, Toronto, ON, Canada, July 17-23, 2016, Proceedings, Part II*, volume 9780 of *Lecture Notes in Computer Science*, pages 422–430. Springer, 2016.

[KJN22] Petri Kotiranta, Marko Junkkari, and Jyrki Nummenmaa. Performance of graph and relational databases in complex queries. *Applied Sciences*, 12(13), 2022.

[KK22] Bas Ketsman and Paraschos Koutris. Modern datalog engines. *Found. Trends Databases*, 12(1):1–68, June 2022.

[KKNV23] Bas Ketsman, Christoph Koch, Frank Neven, and Brecht Vandevoort. Concurrency control for database theorists. *SIGMOD Rec.*, 51(4):6–17, January 2023.

[LFWK09] Senlin Liang, Paul Fodor, Hui Wan, and Michael Kifer. Openrulebench: an analysis of the performance of rule engines. In *Proceedings of the 18th International Conference on World Wide Web*, WWW '09, page 601–610, New York, NY, USA, 2009. Association for Computing Machinery.

[LJA+23] Kyuhwa Lee, Inho Jo, Jungho Ahn, Wontae Sul, and Heon J Jung. Deploying computational storage for htap dbmss takes more than just computation offloading. *Proceedings of the VLDB Endowment*, 16(6):1312–1325, 2023.

[LSIT25] Yanhong A. Liu, Scott D. Stoller, John Idogun, and Yi Tong. Efficiency of analysis of transitive relations using query-driven, ground-and-solve, and fact-driven inference. *CoRR*, abs/2504.21291, 2025.

[MDP23] Hugo Mohr-Daurat and Holger Pirk. Boss - an architecture for database kernel composition. *Proceedings of the VLDB Endowment*, 17(4):739–752, 2023.

[Mel03] Jim Melton. *Advanced SQL:1999: Understanding Object-Relational and Other Advanced Features*. Morgan Kaufmann, 2003.

[MFPR90] I. S. Mumick, S. J. Finkelstein, Hamid Pirahesh, and Raghu Ramakrishnan. Magic is relevant. In *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data*, SIGMOD '90, page 247–258, New York, NY, USA, 1990. Association for Computing Machinery.

[Mon24] MongoDB, Inc. Pymongo - python driver for mongodb. `https://pypi.org/project/pymongo/`, 2024.

[Mon25a] MongoDB. Aggregation pipeline limits. `https://www.mongodb.com/docs/manual/core/aggregation-pipeline-limits/`, 2025.

[Mon25b] MongoDB, Inc., New York, NY. *$graphLookup (expression operator)*, 2025. Accessed: Aug. 27, 2025.

[MS01] Jim Melton and Alan Simon. *SQL:1999: understanding relational language components*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2001.

[Nau89] Jeffrey F. Naughton. Minimizing function-free recursive inference rules. *J. ACM*, 36(1):69–91, January 1989.

[Neo24] Neo4j. Neo4j python driver. `https://neo4j.com/docs/python-manual/current/`, 2024.

[Neo25a] Neo4j. Index-free adjacency. `https://neo4j.com/developer/graph-database-glossary/index-free-adjacency/`, 2025.

[Neo25b] Neo4j. Variable-length patterns. `https://neo4j.com/docs/cypher-manual/current/patterns/variable-length-patterns/`, 2025.

[OB17] João Oliveira and Jorge Bernardino. Newsql databases-memsql and voltdb experimental evaluation. In *KEOD*, pages 276–281, 2017.

[OO14] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (ATC)*, pages 305–319. USENIX Association, 2014.

[Ord05] Carlos Ordonez. Optimizing recursive queries in sql. In *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*, SIGMOD '05, page 834–839, New York, NY, USA, 2005. Association for Computing Machinery.

[Ord10] Carlos Ordonez. Optimization of linear recursive queries in sql. *IEEE Transactions on Knowledge and Data Engineering*, 22(2):264–277, 2010.

[PA16] Andrew Pavlo and Matthew Aslett. What's really new with newsql? *SIGMOD Record*, 45(2):45–55, 2016.

[PHH92] Hamid Pirahesh, Joseph M. Hellerstein, and Waqar Hasan. Extensible/rule based query rewrite optimization in starburst. In *Proceedings of the 1992 ACM SIGMOD International Conference on Management of Data*, SIGMOD '92, page 39–48, New York, NY, USA, 1992. Association for Computing Machinery.

[Pos25] PostgreSQL. With queries (common table expressions). `https://www.postgresql.org/docs/current/queries-with.html`, 2025.

[RM19a] Mark Raasveldt and Hannes Mühleisen. Duckdb: an embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data*, SIGMOD '19, page 1981–1984, New York, NY, USA, 2019. Association for Computing Machinery.

[RM19b] Mark Raasveldt and Hannes Mühleisen. Duckdb: an embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data*, pages 103–116, 2019.

[RQ08] Paolo Romano and Francesco Quaglia. Integration and evaluation of multi-instance-precommit schemes within postgresql. In *Proceedings of the 38th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 314–323. IEEE, 2008.

[RWE15a] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph databases: new opportunities for connected data*. O'Reilly Media, 2015.

[RWE15b] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases: New Opportunities for Connected Data*. O'Reilly Media, 2nd edition, 2015.

[SA16] Panagiotis Sioulas and Anastasia Ailamaki. Vectorizing an in situ query engine. In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)*, pages 2167–2170. ACM, 2016.

[SHP+96] Praveen Seshadri, Joseph M. Hellerstein, Hamid Pirahesh, T. Y. Cliff Leung, Raghu Ramakrishnan, Divesh Srivastava, Peter J. Stuckey, and S. Sudarshan. Cost-based optimization for magic: algebra and implementation. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*, SIGMOD '96, page 435–446, New York, NY, USA, 1996. Association for Computing Machinery.

[Sin24] SingleStore Inc. *WITH (Common Table Expressions)*, 2024.

[Sin25] SingleStore. With (common table expressions). `https://docs.singlestore.com/cloud/query-data/advanced-query-topics/with-common-table-expressions/`, 2025.

[SN17] Galina Shalygina and Boris Novikov. Implementing common table expressions for mariadb. In *Proceedings of the 2nd Conference on Software Engineering and Information Management (SEIM-2017), St. Petersburg, Russia*, volume 21, page 25, 2017.

[SQL99] ISO/IEC 9075-2:1999, Information technology — Database languages — SQL — Part 2: Foundation (SQL/Foundation). International Standard ISO/IEC 9075-2:1999, International Organization for Standardization, Geneva, Switzerland, 1999.

[SW12] Terrance Swift and David s. Warren. Xsb: Extending prolog with tabled logic programming. *Theory Pract. Log. Program.*, 12(1–2):157–187, January 2012.

[SWKH76] Michael Stonebraker, Eugene Wong, Peter Kreps, and Gerald Held. The design and implementation of ingres. *ACM Transactions on Database Systems*, 1(3):189–222, 1976.

[SZ86] Domenico Saccá and Carlo Zaniolo. The generalized counting method for recursive logic queries. In *Proceedings on International Conference on Database Theory*, page 31–53, Berlin, Heidelberg, 1986. Springer-Verlag.

[Taf20] D. Taft. *CockroachDB: The Definitive Guide*. O'Reilly Media, 2020.

[tea25] PyMySQL team. mysqlclient. `https://pypi.org/project/mysqlclient/`, 2025.

[Tek10] Kazim Tuncay Tekle. *Efficient Datalog Queries with Time and Space Complexity Guarantees*. PhD thesis, Computer Science Department, Stony Brook University, 2010.

[TL10] K. Tuncay Tekle and Yanhong A. Liu. Precise complexity analysis for efficient Datalog queries. In *Proceedings of the 12th International ACM SIGPLAN Symposium on Principles and Practice of Declarative Programming*, pages 35–44, 2010.

[TL11] K. Tuncay Tekle and Yanhong A. Liu. More efficient datalog queries: subsumptive tabling beats magic sets. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data*, SIGMOD '11, page 661–672, New York, NY, USA, 2011. Association for Computing Machinery.

[TPLH16] Junichi Tatemura, Oliver Po, Zheng Li, and Hakan Hacigümüş. Strudel: A framework for transaction performance analyses on sql/nosql systems. In *Proceedings of the 19th International Conference on Extending Database Technology (EDBT)*, pages 534–537, 2016.

[TSM+20] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, et al. Cockroachdb: The resilient geo-distributed sql database. *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1493–1509, 2020.

[Ull88] Jeffrey D. Ullman. *Principles of database and knowledge-base systems, Vol. I.* Computer Science Press, Inc., USA, 1988.

[Ull90] Jeffrey D. Ullman. *Principles of Database and Knowledge-Base Systems: Volume II: The New Technologies*. W. H. Freeman & Co., USA, 1990.

[VMZ+10a] Chad Vicknair, Michael Macias, Zhendong Zhao, Xiaofei Nan, Yixin Chen, and Dawn Wilkins. A comparison of a graph database and a relational database: a data provenance perspective. In *Proceedings of the 48th Annual ACM Southeast Conference*, ACMSE '10, New York, NY, USA, 2010. Association for Computing Machinery.

[VMZ+10b] Chad Vicknair, Michael Macias, Zhendong Zhao, Xiaofei Nan, Yixin Chen, and Dawn Wilkins. A comparison of a graph database and a relational database: a data provenance perspective. In *Proceedings of the 48th Annual ACM Southeast Conference*, ACMSE '10, New York, NY, USA, 2010. Association for Computing Machinery.

[War92] David S. Warren. Memoing for logic programs. *Commun. ACM*, 35(3):93–111, March 1992.

[Web12] Jim Webber. *A programmatic introduction to Neo4j*. Addison-Wesley Professional, 2012.

[WZ02] Haixun Wang and Carlo Zaniolo. The ATLaS system and its powerful database language based on simple extensions of SQL. In *Proceedings of the 18th International Conference on Data Engineering (ICDE)*, pages 494–505. IEEE, 2002.

[ZHNB07] Marcin Zukowski, Sándor Héman, Niels Nes, and Peter Boncz. Cooperative scans: dynamic bandwidth sharing in a dbms. In *Proceedings of the 33rd International Conference on Very Large Data Bases*, VLDB '07, page 723–734. VLDB Endowment, 2007.

[ZMB17] Yiheng Zhu, Wenjing Ma, and Yungang Bao. Acia, not acid: conditions, properties and challenges. *arXiv preprint arXiv:1701.07512*, 2017.